

**PENGEMBANGAN ARSITEKTUR MULTI-AGENT
SYSTEM BERBASIS LLM FUNCTION CALLING
UNTUK OTOMASI TRIASE KELUHAN DI FASILITAS
HUNIAN MAHASISWA KAMPUS XYZ**

**ADITYA FAJRI
2602113205**



**PROGRAM STUDI MAGISTER TEKNIK INFORMATIKA
BINUS GRADUATE PROGRAM
UNIVERSITAS BINA NUSANTARA
JAKARTA
2026**

**PENGEMBANGAN ARSITEKTUR MULTI-AGENT
SYSTEM BERBASIS LLM FUNCTION CALLING
UNTUK OTOMASI TRIASE KELUHAN DI FASILITAS
HUNIAN MAHASISWA KAMPUS XYZ**

**ADITYA FAJRI
2602113205**



Tesis
Sebagai Salah Satu Syarat untuk Memperoleh Gelar
Magister Teknik Informatika
pada

**PRORGRAM STUDI MAGISTER TEKNIK INFORMATIKA
BINUS GRADUATE PROGRAM
UNIVERSITAS BINA NUSANTARA
JAKARTA
2026**

DAFTAR ISI

DAFTAR ISI	iii
DAFTAR TABEL	iv
DAFTAR GAMBAR	v
DAFTAR RUMUS	vi
1. BAB I PENDAHULUAN	1
1.1. Latar Belakang	1
1.2. Perumusan Masalah	2
1.3. Tujuan Penelitian	3
1.4. Manfaat Penelitian	3
1.5. Ruang Lingkup Penelitian	4
2. BAB II TINJAUAN PUSTAKA	5
2.1. <i>Large Language Models</i> (LLM)	5
2.2. <i>Supervised Fine-Tuning</i> (SFT)	6
2.3. <i>Cloud-based Model Adaptation</i> melalui <i>Vertex AI</i>	8
2.4. <i>Single Agent</i> versus <i>Multi-Agent System</i> (MAS)	10
2.5. <i>Function Calling</i> pada <i>Large Language Models</i>	11
2.6. <i>Application Programming Interface</i> (API) dan <i>Webhook</i>	13
2.7. <i>Retrieval-Augmented Generation</i> (RAG)	15
2.8. Otomatisasi Triase Keluhan Fasilitas Hunian	16
2.9. Tinjauan Penelitian Terdahulu	18
3. BAB III METODOLOGI PENELITIAN	26
3.1. Kerangka Berpikir	26
3.2. Tahapan Penelitian	28
3.3. Pengumpulan & Persiapan Data	30
3.4. Skenario <i>Single Agent</i> (Sistem Pembanding)	31
3.5. Agen 1: Penerima Pesan & RAG	32
3.6. Agen 2: Triase & Klasifikasi	33
3.7. Agen 3: Eksekutor Tiket	34
3.8. Orkestrasi dan Integrasi Multi-Agent System	35
3.9. Metode Evaluasi	37
3.10. Jadwal Penelitian	41
DAFTAR PUSTAKA	43

DAFTAR TABEL

Tabel 2.1 Contoh Schema JSON	13
Tabel 2.2 Tabel Komparasi Penelitian Terdahulu	20
Tabel 3.1 Contoh Schema JSON	34
Tabel 3.2 Rincian Jadwal Kegiatan Penelitian	41

DAFTAR GAMBAR

Gambar 2.1 Large Language Models	5
Gambar 3.1 Alur Pemikiran Penelitian	27
Gambar 3.2 Alur Tahapan Penelitian	29
Gambar 3.3 Alur Single Agent	32
Gambar 3.5 Alur Multi-Agent System	37

DAFTAR RUMUS

Rumus 3.1: Akurasi.....	38
Rumus 3.2: Validitas JSON	38
Rumus 3.3: Tingkat Halusinasi	39
Rumus 3.4: Efisiensi Pemakaian Komputasi	39
Rumus 3.5: Efisiensi Token	40
Rumus 3.6: Mean Time To Resolve (MTTR).....	40

BAB I

PENDAHULUAN

1.1. Latar Belakang

Pengelolaan fasilitas hunian modern, khususnya asrama dan apartemen mahasiswa, kini menghadapi tuntutan efisiensi layanan yang semakin meningkat. Standar ekspektasi penghuni telah bergeser secara signifikan. Saat ini, penghuni tidak hanya memerlukan fasilitas fisik yang memadai, tetapi juga mengharapkan respons layanan yang cepat dan tanggap. Di lingkungan universitas, efektivitas penanganan kerusakan serta bantuan teknis menjadi parameter utama untuk menilai kualitas operasional pengelola fasilitas.

Kesenjangan efisiensi ini terlihat nyata pada pengelolaan hunian mahasiswa di Kampus XYZ. Pihak pengelola bertanggung jawab penuh atas empat gedung yang mencakup hunian putra, hunian putri, serta penginapan tamu. Namun, seluruh operasional harian untuk keempat gedung tersebut hanya dikelola oleh satu hingga dua orang staf resepsionis secara bergantian. Keterbatasan sumber daya manusia ini menciptakan tekanan kerja yang tinggi. Staf harus menyelesaikan berbagai tugas administratif secara simultan, mulai dari melayani tamu dan menjelaskan aturan hunian, hingga melakukan triase manual terhadap setiap keluhan yang masuk.

Hambatan operasional tersebut bersumber dari perbedaan format komunikasi antara mahasiswa dan sistem manajemen fasilitas. Mahasiswa cenderung menghindari aplikasi pelaporan berbasis web yang kaku karena mengharuskan pengisian formulir secara mandiri. Sebagai alternatif, mereka lebih menyukai jalur komunikasi informal melalui pesan instan, seperti WhatsApp, untuk menghubungi staf secara langsung. Kecenderungan ini menyebabkan proses pencatatan tidak berjalan otomatis dan sangat bergantung pada ketelitian staf. Petugas harus memilah jenis kerusakan, menentukan tingkat urgensi, serta mencari departemen teknis yang tepat sebelum memasukkannya ke portal pelaporan. Proses konversi pesan teks tidak terstruktur menjadi data formal ini memakan banyak waktu dan rentan terhadap kesalahan pengelompokan, terutama saat volume laporan meningkat.

Guna mengurangi beban administratif tersebut, manajemen fasilitas memerlukan solusi otomatisasi berbasis kecerdasan buatan. Perkembangan *Large Language Models (LLM)* menawarkan kemampuan pemrosesan bahasa alami yang kuat untuk mengatasi kendala ini. Saat ini, *LLM* telah berkembang menjadi agen virtual yang mampu menjalankan tindakan eksternal melalui mekanisme *Function Calling*. Fitur tersebut memungkinkan model bahasa mengidentifikasi variabel dari percakapan dan menghasilkan instruksi teknis dalam format *JSON*. Melalui mekanisme ini, *LLM* dapat menerjemahkan laporan informal mahasiswa menjadi *payload* yang sesuai dengan *Application Programming Interface (API)* pelaporan di Kampus XYZ.

Namun, penerapan arsitektur *LLM* untuk tugas kompleks menghadapi kendala teknis jika menggunakan pendekatan *Single Agent*. Pada arsitektur ini, satu model tunggal menangani seluruh alur kerja, mulai dari memahami konteks hingga eksekusi data. Berbagai literatur menunjukkan bahwa agen tunggal rentan mengalami *context window overflow* serta peningkatan tingkat halusinasi (Sun et al., 2025) ketika memproses beban kerja yang terlalu beragam. Sebagai solusi, pendekatan *Multi-Agent System (MAS)* mengurangi risiko tersebut dengan mendistribusikan tugas kepada beberapa agen spesialis. Pembagian peran ini memastikan setiap entitas agen hanya memproses informasi yang relevan dengan fungsinya. Hasilnya, manajemen memori sistem menjadi lebih optimal dan keandalan keputusan meningkat (Li et al., 2024).

Berdasarkan permasalahan tersebut, penelitian ini merancang, menguji, dan membandingkan kinerja implementasi arsitektur *Multi-Agent System (MAS)* dan *Single Agent*. Evaluasi ini berfokus pada otomatisasi triase keluhan di fasilitas hunian mahasiswa dengan mengukur akurasi teknis model serta dampaknya terhadap efisiensi operasional.

1.2. Perumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, masalah dalam penelitian ini dirumuskan ke dalam pertanyaan-pertanyaan berikut:

1. Bagaimana merancang dan membangun arsitektur *Multi-Agent System* (MAS) berbasis LLM *function calling* untuk mengotomatisasi triase keluhan di fasilitas hunian Kampus XYZ?
2. Bagaimana perbandingan kinerja antara arsitektur MAS dan *Single Agent* dalam mengklasifikasikan keluhan, ditinjau dari metrik akurasi, tingkat halusinasi (*hallucination rate*), dan validitas format JSON yang dihasilkan?
3. Bagaimana tingkat efisiensi komputasi dan operasional dari kedua arsitektur tersebut, dievaluasi berdasarkan konsumsi *token*, *latency*, serta estimasi dampaknya terhadap penurunan *Mean Time To Resolve* (MTTR)?

1.3. Tujuan Penelitian

Penelitian ini bertujuan untuk menjawab rumusan masalah melalui sasaran-sasaran berikut:

1. Merancang dan membangun arsitektur *Multi-Agent System* (MAS) yang mendayagunakan LLM *function calling* untuk mengotomatisasi alur pelaporan dan triase keluhan.
2. Mengevaluasi perbandingan performa arsitektur MAS dan *Single Agent* melalui pengukuran empiris terhadap akurasi klasifikasi, tingkat halusinasi, serta keabsahan *payload* JSON sesuai skema API manajemen.
3. Mengukur efisiensi sistem melalui kalkulasi penggunaan *token* dan *latency*, serta memproyeksikan dampaknya terhadap percepatan waktu penanganan masalah (MTTR).

1.4. Manfaat Penelitian

Penelitian ini bertujuan untuk menjawab rumusan masalah melalui sasaran-sasaran berikut:

1. Memberikan kontribusi pada literatur sistem multi-agen berbasis LLM. Penelitian ini menyajikan bukti empiris bahwa pembagian peran antar-agen dapat secara signifikan mengurangi halusinasi informasi saat memproses teks bahasa alami yang beragam.
2. Menjadi acuan teknis dalam pemanfaatan agen cerdas untuk otomasi tugas administratif. Secara spesifik, penelitian ini mendemonstrasikan

kemampuan *function calling* dalam mengubah pesan teks informal menjadi data instruksi yang terstruktur.

3. Menyediakan arsitektur sistem yang siap pakai bagi pengelola fasilitas. Sistem ini mengoptimalkan infrastruktur yang ada dengan mengintegrasikan pengetahuan dari *Handbook* ke dalam API berbasis .NET milik kampus.
4. Mengurangi beban kerja dan kelelahan kognitif staf administrasi akibat proses pencatatan manual. Efisiensi ini secara langsung akan mempercepat waktu tanggap layanan (MTTR) dan meningkatkan kepuasan penghuni.

1.5. Ruang Lingkup Penelitian

Ruang lingkup dalam penelitian ini dapat dijabarkan sebagai berikut:

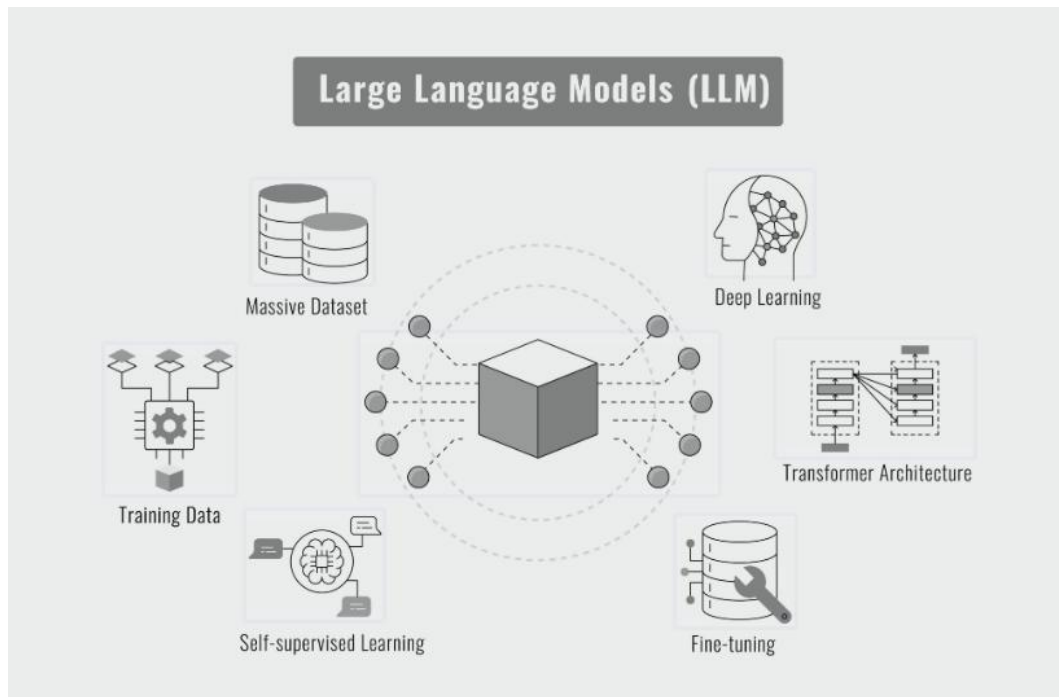
1. Sistem diuji menggunakan rekonstruksi data riwayat keluhan dari fasilitas hunian Kampus XYZ. Pemrosesan teks secara khusus difokuskan pada interaksi bahasa Indonesia sehari-hari (informal).
2. Basis pengetahuan (*knowledge base*) yang digunakan sistem untuk memastikan kebenaran informasi murni berasal dari buku panduan resmi (*Handbook*). Metode pencarian informasi ini menggunakan *Retrieval-Augmented Generation* (RAG).
3. Sistem dikembangkan menggunakan model komersial Google Gemini yang diakses melalui lingkungan *Vertex AI*. Model ini disesuaikan perilakunya melalui *Supervised Fine-Tuning* (SFT) menggunakan data set berformat .jsonl.
4. Integrasi teknis dibatasi pada pengiriman instruksi sistem (*function calling*) melalui *webhook*. Data tersebut dikirim langsung ke *endpoint* API pelaporan berbasis .NET milik manajemen fasilitas.
5. Metode evaluasi keandalan hanya berfokus pada metrik komputasi, yaitu akurasi klasifikasi, halusinasi, validitas JSON, konsumsi *token*, dan *latency*. Penelitian ini tidak mengukur kualitas pengerjaan perbaikan fisik di lapangan.

BAB II

TINJAUAN PUSTAKA

2.1. *Large Language Models (LLM)*

Large Language Model (LLM) merupakan model kecerdasan buatan yang dirancang untuk memproses, memahami, dan menghasilkan teks bahasa manusia dengan tingkat akurasi yang tinggi. Secara teknis, model ini termasuk dalam kategori jaringan saraf tiruan mendalam (*deep neural network*) yang dilatih menggunakan metode *self-supervised learning* pada kumpulan data teks yang sangat besar. Melalui proses pelatihan ini, LLM belajar mengenali pola bahasa, struktur kalimat, hingga keterkaitan antar konsep dalam berbagai konteks komunikasi.



Gambar 2.1 Large Language Models

Arsitektur utama yang menjadi fondasi bagi perkembangan LLM saat ini adalah *Transformer*. Arsitektur ini pertama kali diperkenalkan oleh Vaswani et al. (2017) dan menjadi basis bagi berbagai model populer seperti GPT, LLaMA, Gemini, dan Claude. Komponen inti dalam *Transformer* adalah mekanisme *self-attention*. Mekanisme ini memungkinkan model untuk menghitung tingkat relevansi antara satu kata dengan kata lainnya dalam sebuah kalimat secara

bersamaan. Dengan kemampuan ini, model dapat memahami hubungan kata yang berjauhan dalam satu paragraf panjang, sehingga makna yang dihasilkan tetap konsisten dan utuh. Keunggulan tersebut membuat LLM berbeda dari mesin pencari konvensional. LLM tidak sekadar mencari kata kunci, tetapi memahami maksud sebenarnya di balik permintaan pengguna (Li et al., 2024).

Perkembangan penting berikutnya dalam evolusi teknologi ini adalah munculnya kemampuan *instruction following*. Fitur ini memungkinkan model untuk mematuhi perintah yang kompleks dan bertahap dalam bahasa sehari-hari. Kemampuan tersebut mengubah posisi LLM dari sekadar alat bantu percakapan (*chatbot*) menjadi fondasi utama bagi agen virtual (*AI agents*). Agen virtual merupakan sistem berbasis LLM yang memiliki kemampuan tambahan untuk merencanakan tindakan, menggunakan perangkat eksternal, dan berinteraksi dengan sistem di luar dirinya sendiri (Xu et al., 2025). Dalam penelitian ini, LLM bertindak sebagai unit pemroses utama yang bertugas memahami laporan keluhan penghuni fasilitas hunian. Model akan mengolah bahasa sehari-hari yang sering kali tidak baku menjadi instruksi teknis yang terstruktur agar dapat terbaca oleh sistem manajemen fasilitas.

Riset terbaru memperkuat posisi LLM sebagai solusi efektif untuk tugas-tugas yang memerlukan pemahaman konteks mendalam. Jiang et al. (2024) menjelaskan bahwa integrasi LLM dalam sistem komunikasi tingkat lanjut mampu memproses banyak instruksi dari berbagai sumber sekaligus serta menghasilkan keputusan yang adaptif. Di sisi lain, Rivkin et al. (2025) membuktikan bahwa LLM dapat berfungsi sebagai unit kendali otonom yang mengatur berbagai perangkat dalam lingkungan rumah pintar (*smart home*). Temuan ini sangat relevan dengan kebutuhan otomatisasi operasional di fasilitas hunian, di mana sistem harus mampu merespons laporan kerusakan atau keluhan secara cepat dan tepat sasaran melalui pemahaman bahasa alami yang baik.

2.2. *Supervised Fine-Tuning* (SFT)

Supervised Fine-Tuning (SFT) merupakan proses adaptasi model bahasa besar untuk menyelaraskan perilaku model dengan pola instruksi dan respons pada domain tertentu. Tahapan ini sangat penting karena model dasar hasil pelatihan

awal (*pre-training*) biasanya hanya belajar memprediksi kata berikutnya dari data internet yang sangat luas. SFT bekerja dengan cara melatih kembali model menggunakan kumpulan data yang berisi pasangan instruksi dan jawaban ideal yang telah dipilih secara manual. Chung et al. (2024) menjelaskan bahwa proses penyesuaian instruksi ini meningkatkan kemampuan model dalam mengikuti perintah yang rumit secara signifikan. Hasil penelitian mereka menunjukkan bahwa model dapat merespons pertanyaan dari bidang khusus dengan lebih baik meskipun jumlah data pelatihan yang digunakan jauh lebih sedikit daripada saat tahap pra-pelatihan.

Penggunaan SFT dalam penelitian ini bertujuan untuk mengatasi kebutuhan spesifik dalam memahami konteks operasional pada fasilitas hunian Kampus XYZ. Model seperti Gemini API memang memiliki kemampuan bahasa umum yang sangat kuat. Namun, model tersebut tetap memerlukan pemahaman mendalam mengenai istilah teknis bangunan, struktur departemen pengelola, serta aturan pelaporan yang berlaku di lingkungan kampus. SFT membantu sistem untuk menyerap informasi khusus ini ke dalam parameter model. Melalui proses ini, model dapat menghasilkan jawaban yang tidak hanya benar secara tata bahasa, tetapi juga akurat secara operasional sesuai dengan keadaan di lapangan.

Kualitas data instruksi memegang peranan kunci dalam keberhasilan proses penyesuaian model. Wang et al. (2023) memperkenalkan kerangka kerja *Self-Instruct* yang membuktikan bahwa instruksi berkualitas tinggi dapat meningkatkan kemampuan model dalam mengikuti berbagai perintah. Peneliti menerapkan prinsip tersebut untuk menyusun skenario keluhan yang mencakup berbagai gaya bahasa. Data pelatihan tidak hanya berisi laporan dalam bahasa formal, tetapi juga mencakup variasi bahasa informal, kata-kata gaul, serta pencampuran bahasa (*code-switching*) yang sering digunakan oleh mahasiswa. Strategi ini bertujuan untuk memperkecil celah antara cara komunikasi mahasiswa yang tidak baku dengan format data terstruktur yang dibutuhkan oleh sistem manajemen fasilitas.

Implementasi teknis SFT dalam penelitian ini dilakukan menggunakan lingkungan *Vertex AI* di dalam *Google Cloud Platform*. Peneliti memilih pendekatan berbasis awan (*cloud*) karena menawarkan fleksibilitas serta kemampuan penanganan beban kerja yang lebih baik daripada perangkat keras

lokal. Peneliti mengelola seluruh rangkaian kegiatan, mulai dari pengiriman dataset dalam format *.jsonl* hingga pemantauan percobaan, melalui antarmuka *Google AI Studio*. Penggunaan infrastruktur ini memungkinkan penelitian berjalan dengan kapasitas komputasi skala industri. Hal tersebut membantu peneliti menghindari hambatan keterbatasan perangkat keras yang sering muncul di lingkungan akademik, sekaligus menjaga agar setiap percobaan dapat diulang dengan hasil yang konsisten.

SFT memberikan keunggulan strategis karena mampu membentuk perilaku model tanpa menghilangkan kemampuan dasar yang sudah ada. Proses ini tetap mempertahankan pemahaman bahasa umum dari tahap pra-pelatihan, namun menambahkan lapisan pemahaman baru yang sangat spesifik pada bidang hunian mahasiswa. Keseimbangan ini memastikan sistem tetap mampu menangani percakapan sehari-hari dengan baik. Model dapat memproses keluhan fasilitas, seperti kerusakan pendingin ruangan yang disampaikan dengan bahasa santai, hingga pertanyaan resmi mengenai aturan kunjungan. Hasilnya, sistem dapat memberikan tanggapan yang konsisten dan akurat dalam satu kerangka model yang terpadu.

2.3. *Cloud-based Model Adaptation* melalui *Vertex AI*

Cloud-based Model Adaptation merupakan sebuah metode pengembangan sistem kecerdasan buatan yang memindahkan seluruh proses pelatihan, penyesuaian, dan penggunaan model ke dalam infrastruktur awan. Pendekatan ini menjadi solusi bagi peneliti yang menghadapi kendala biaya tinggi dan kerumitan teknis saat harus mengelola perangkat keras seperti GPU secara mandiri. Melalui metode ini, beban kerja komputasi yang berat dialihkan ke penyedia layanan pihak ketiga. Dalam penelitian ini, pengembang menjalankan seluruh siklus adaptasi model mulai dari *Supervised Fine-Tuning* (SFT) hingga tahap penyajian model (*serving*) melalui API di atas platform *Vertex AI*. Google Cloud menyediakan platform ini untuk mengelola beban kerja pembelajaran mesin secara otomatis sehingga peneliti dapat lebih fokus pada pengembangan model daripada pemeliharaan infrastruktur.

Vertex AI memberikan keuntungan mendasar yang sangat mendukung kebutuhan riset ini. Peneliti memilih platform ini karena menyediakan akses langsung ke *Gemini API Model*. Akses ini sangat penting karena memungkinkan penggunaan model bahasa yang sudah dilatih menggunakan data yang sangat besar, yang tidak mungkin dilakukan melalui riset akademik secara mandiri. Selain itu, Google AI Studio menyediakan satu lingkungan kerja yang terpadu. Fasilitas ini mempermudah peneliti dalam mengelola kumpulan data, mengatur konfigurasi percobaan SFT, serta memantau perkembangan hasil pelatihan secara langsung. Platform ini juga menggunakan infrastruktur *serverless*. Hal ini menjamin bahwa proses penyesuaian model tidak akan terganggu oleh keterbatasan kapasitas perangkat keras tertentu, sehingga peneliti lain dapat mengulang eksperimen ini dengan hasil yang tetap konsisten.

Secara teknis, proses adaptasi model berbasis awan dalam penelitian ini mengikuti beberapa langkah yang saling berkaitan. Peneliti mengawali proses dengan mengirimkan kumpulan data SFT dalam format *.jsonl* ke repositori *Cloud Storage*. Data tersebut disusun dalam bentuk pasangan pertanyaan dari penghuni fasilitas hunian dan jawaban benar yang telah divalidasi sebagai referensi utama bagi model. Setelah data tersedia, peneliti mengatur konfigurasi eksperimen melalui konsol Vertex AI. Pengaturan ini mencakup penentuan angka-angka teknis seperti jumlah langkah pelatihan, ukuran kelompok data (*batch size*), dan kecepatan pembelajaran (*learning rate*). Setelah proses pelatihan selesai, peneliti menerapkan model tersebut sebagai sebuah titik akhir (*endpoint*) khusus. Titik akhir ini berfungsi agar lapisan sistem agen dapat mengakses kemampuan model melalui panggilan API standar. Seluruh tahapan ini mengikuti standar praktik terbaik *Machine Learning Operations* (MLOps) untuk menjaga keteraturan sistem.

Penerapan *Cloud-based Model Adaptation* sebenarnya membawa perubahan yang lebih mendalam daripada sekadar pindah lokasi penyimpanan data. Paradigma ini mengubah cara peneliti berinteraksi dengan model dasar (*foundation model*). Peneliti tidak lagi mengelola bobot model secara manual satu demi satu, melainkan mengendalikan perilaku model melalui pengelolaan data dan pengaturan pelatihan yang sistematis. Perubahan ini memberikan tingkat abstraksi yang lebih tinggi dan sangat bermanfaat bagi penelitian terapan. Efisiensi dalam proses

pengembangan serta keberlanjutan sistem untuk jangka panjang menjadi prioritas utama. Dengan demikian, peneliti dapat menghasilkan sistem yang stabil dan mudah dikelola tanpa mengabaikan ketepatan teknis dari model tersebut.

2.4. *Single Agent versus Multi-Agent System (MAS)*

Single Agent merupakan arsitektur kecerdasan buatan yang mengandalkan satu unit model tunggal untuk menjalankan seluruh rangkaian tugas secara mandiri. Dalam sistem ini, model memikul tanggung jawab penuh mulai dari memproses pesan pengguna, mencari data pada dokumen referensi, hingga mengambil keputusan untuk tindakan selanjutnya. Arsitektur ini cukup populer karena proses pengembangannya yang cenderung sederhana dan cepat. Namun, ketergantungan pada satu entitas pusat menciptakan kendala teknis yang signifikan saat sistem harus menangani beban kerja yang bertambah besar dan bervariasi.

Salah satu batasan utama pada *Single Agent* berkaitan dengan kapasitas ruang memori atau *context window*. Model bahasa memiliki batas jumlah teks yang dapat ia proses dalam satu waktu. Ketika satu agen harus mengelola riwayat percakapan yang panjang, membaca dokumen panduan hunian yang sangat tebal, dan menjalankan instruksi teknis secara bersamaan, kapasitas memori ini akan cepat penuh. Kondisi tersebut memaksa agen untuk membuang informasi lama demi memproses data baru, yang sering kali menyebabkan hilangnya konteks penting. Selain itu, beban kerja yang bertumpuk pada satu titik meningkatkan risiko terjadinya halusinasi, yaitu kondisi di mana model memberikan jawaban yang tampak benar namun sebenarnya salah (Sun et al., 2025).

Sebagai alternatif, *Multi-Agent System (MAS)* hadir dengan pendekatan yang mendistribusikan beban kerja tersebut. MAS membagi tugas besar menjadi beberapa bagian kecil yang dikerjakan oleh agen-agen dengan peran khusus. Pembagian ini bertujuan untuk menjaga agar setiap agen tetap fokus pada satu fungsi tertentu. Misalnya, satu agen bertugas hanya untuk melayani komunikasi dengan penghuni, sementara agen lainnya fokus mencari informasi pada dokumen aturan hunian. Agen ketiga kemudian berperan khusus untuk mengubah hasil diskusi tersebut menjadi data terstruktur yang masuk ke dalam sistem pelaporan. Dengan pembagian ini, setiap agen hanya perlu memproses informasi yang relevan

dengan tugasnya, sehingga penggunaan *context window* menjadi lebih hemat dan akurat.

Koordinasi antar agen dalam MAS memastikan bahwa setiap bagian sistem bekerja secara terpadu tanpa saling tumpang tindih. Mekanisme ini menyelesaikan masalah kompleksitas yang tidak mampu ditangani oleh agen tunggal. Penelitian oleh Jiang et al. (2024) menunjukkan bahwa pembagian kerja dalam arsitektur multi-agen mampu menghasilkan keputusan yang jauh lebih efisien pada sistem komunikasi yang rumit. Senada dengan temuan tersebut, Zhong et al. (2024) membuktikan melalui sistem CASIT bahwa pemisahan peran antar agen mampu meningkatkan ketepatan pengelompokan data dan mempercepat waktu tanggap sistem secara keseluruhan.

Penerapan MAS juga menjadi solusi atas keterbatasan fisik dari model bahasa itu sendiri. Li et al. (2024) menegaskan bahwa koordinasi banyak agen merupakan kunci utama untuk mengatasi batas memori pemrosesan dan meningkatkan keandalan sistem secara menyeluruh. Melalui pendekatan ini, risiko kegagalan sistem akibat kelebihan informasi dapat dikurangi secara signifikan. Penggunaan arsitektur MAS dalam penelitian ini menjadi landasan yang kuat untuk membangun sistem otomatisasi triase yang akurat, stabil, dan mampu menangani beragam keluhan fasilitas hunian dengan lebih baik daripada model agen tunggal.

2.5. *Function Calling pada Large Language Models*

Function calling merupakan fitur pada *Large Language Models* (LLM) yang memungkinkan model untuk mengidentifikasi kebutuhan akan tindakan eksternal dan menghasilkan instruksi terstruktur untuk menjalankan fungsi tersebut. Kemampuan ini memperluas peran LLM yang awalnya hanya berfungsi sebagai generator teks menjadi sebuah pengendali sistem yang aktif. Melalui mekanisme ini, LLM dapat menjembatani celah antara komunikasi manusia yang tidak terstruktur dengan sistem komputasi yang memerlukan format data kaku. Hal ini sangat penting untuk mengatasi keterbatasan LLM dalam melakukan tugas-tugas spesifik seperti perhitungan matematika yang kompleks, pengambilan data waktu nyata, atau interaksi dengan perangkat lunak pihak ketiga.

Dalam literatur akademik, konsep ini sering disebut juga sebagai *tool use* atau *API invocation*. Menurut Xu et al. (2025), *function calling* adalah kemampuan LLM untuk secara proaktif memutuskan kapan sebuah alat eksternal harus digunakan dan bagaimana cara mengoperasikannya dengan tepat. Secara teknis, pengembang terlebih dahulu mendefinisikan *function schema* dalam sebuah spesifikasi formal. Spesifikasi ini mencakup nama fungsi, deskripsi parameter yang dibutuhkan, serta tipe data untuk setiap parameter tersebut. Pengembang kemudian memasukkan *schema* ini ke dalam *system prompt* yang menjadi panduan dasar bagi LLM.

Proses kerja *function calling* dimulai ketika LLM menerima masukan dari pengguna dan menentukan bahwa respon yang dibutuhkan melibatkan tindakan di luar kemampuan internalnya. LLM tidak mengeksekusi tindakan tersebut secara langsung ke pangkalan data atau API. Sebaliknya, LLM akan menghasilkan keluaran dalam format *JavaScript Object Notation* (JSON) yang terstruktur. Format JSON ini berisi informasi mengenai fungsi mana yang harus dipanggil beserta parameter spesifik yang telah diekstraksi dari percakapan pengguna. Sistem eksternal kemudian bertugas membaca JSON tersebut untuk menjalankan perintah nyata di lingkungan perangkat lunak yang dituju.

Sebagai contoh pada sistem otomasi pelaporan masalah di hunian, ketika seorang penghuni mengirimkan pesan mengenai lampu di lorong lantai tiga yang padam, LLM tidak hanya memberikan balasan berupa teks simpati. Model tersebut mengenali bahwa informasi tersebut memenuhi syarat untuk pembuatan tiket perbaikan. Berdasarkan instruksi yang telah diberikan, LLM mengekstraksi variabel penting seperti kategori kelistrikan, lokasi di lantai tiga, dan tingkat urgensi sedang. Hasil ekstraksi ini dituangkan dalam sebuah objek JSON terstruktur yang ditujukan pada fungsi pembuatan tiket perbaikan. Dengan demikian, sistem dapat langsung mencatat laporan tersebut ke dalam database tanpa memerlukan staf manusia untuk menginput data secara manual.

```

{
  "function": "create_maintenance_ticket",
  "parameters": {
    "category": "Kelistrikan",
    "location": "Lantai 3 - Lorong",
    "description": "Lampu lorong padam",
    "urgency": "Sedang",
    "target_department": "Tim Teknik"
  }
}

```

Tabel 2.1 Contoh Schema JSON

Penerapan teknologi ini telah teruji dalam berbagai skenario operasional yang memerlukan ketepatan tinggi. Riset yang dilakukan oleh Rivkin et al. (2025) pada ekosistem rumah pintar berbasis *Artificial Intelligence of Things* (AIoT) memberikan bukti mengenai keandalan mekanisme ini. Dalam penelitian tersebut, LLM terbukti mampu menerjemahkan instruksi bahasa alami dari penghuni menjadi panggilan fungsi yang akurat untuk mengendalikan perangkat rumah tangga. Keberhasilan ini menunjukkan bahwa *function calling* merupakan komponen inti yang memungkinkan LLM berfungsi sebagai pusat kendali dalam arsitektur sistem berbasis agen yang otonom.

2.6. Application Programming Interface (API) dan Webhook

Application Programming Interface (API) merupakan sekumpulan aturan dan protokol yang memungkinkan satu aplikasi perangkat lunak untuk berinteraksi dengan aplikasi lainnya secara teratur. Dalam struktur sistem komputer, API berfungsi sebagai perantara yang menghubungkan berbagai komponen perangkat lunak agar dapat bertukar informasi tanpa harus memahami kerumitan internal masing-masing sistem. Penggunaan API bertujuan untuk standarisasi komunikasi data sehingga integrasi antar sistem yang berbeda dapat dilakukan dengan lebih mudah dan efisien. Pada perkembangan terbaru, API berbasis *Representational State Transfer* (REST) yang beroperasi di atas protokol HTTP menjadi standar yang

paling banyak digunakan karena sifatnya yang ringan dan memiliki keterpaduan yang baik dengan berbagai bahasa pemrograman (Xu et al., 2025).

Keterkaitan antara mekanisme *function calling* pada model bahasa besar (*Large Language Model* atau LLM) dengan API merupakan bagian dasar dalam arsitektur penelitian ini. Saat agen LLM menghasilkan keluaran data dalam format JSON melalui *function calling*, data tersebut memerlukan saluran komunikasi untuk dapat dieksekusi. Di sinilah API berperan sebagai jembatan yang menerima data terstruktur tersebut untuk diteruskan ke sistem tujuan. Lapisan *orchestration* dalam sistem akan membaca data JSON tersebut, kemudian menggunakannya sebagai isi pesan (*payload*) untuk melakukan permintaan HTTP (*request*) menuju *endpoint* API yang relevan. Dalam konteks penelitian ini, sistem memanfaatkan API dari portal manajemen fasilitas berbasis .NET yang telah beroperasi di lingkungan kampus. API ini bertugas menerima data, melakukan validasi, serta menjalankan logika bisnis pada sisi peladen (*server*), seperti menyimpan tiket laporan ke dalam basis data dan mengirimkan konfirmasi kepada sistem agen.

Selain API, penelitian ini juga melibatkan *Webhook* untuk menciptakan komunikasi dua arah yang lebih responsif. *Webhook* adalah mekanisme pengiriman data berbasis kejadian (*event-driven*) yang bekerja dengan cara yang berbeda dari API. Jika API biasa umumnya bersifat menarik data (*pull*), di mana klien harus terus-menerus bertanya kepada peladen untuk mendapatkan pembaruan, maka *Webhook* bekerja dengan cara mendorong data (*push*). Server akan secara otomatis mengirimkan informasi ke alamat URL tertentu segera setelah suatu peristiwa terjadi. Mekanisme ini menyelesaikan masalah keterlambatan informasi dan mengurangi beban kerja sistem karena klien tidak perlu melakukan permintaan data secara berulang kali.

Integrasi antara API dan *Webhook* sangat penting dalam proses otomatisasi triase keluhan fasilitas hunian. Dalam skenario ini, API digunakan untuk mengirimkan laporan awal dari mahasiswa ke sistem manajemen, sementara *Webhook* bertugas mengirimkan pembaruan status secara langsung dari sistem manajemen kembali ke agen LLM. Sebagai contoh, ketika tim teknis mengubah status laporan menjadi "sedang dikerjakan" pada sistem pusat, *Webhook* akan segera memicu notifikasi otomatis. Agen kemudian menerima informasi tersebut

dan meneruskannya kembali kepada mahasiswa pelapor. Gabungan kedua teknologi ini memungkinkan terciptanya siklus komunikasi yang tertutup dan mandiri. Alur kerja yang otomatis ini meminimalkan keterlibatan manual dari staf administrasi, sehingga proses penanganan keluhan menjadi lebih cepat dan akurat.

2.7. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) merupakan sebuah kerangka kerja arsitektur yang dirancang untuk meningkatkan akurasi *Large Language Model* (LLM) dengan cara mengintegrasikan data luar yang tidak ada dalam data pelatihan asli model tersebut. Secara mendasar, RAG menjawab keterbatasan utama LLM yang bersifat statis dan sering kali memberikan informasi yang tidak akurat atau halusinasi saat menangani permintaan yang sangat spesifik. Arsitektur ini memungkinkan sistem untuk mencari dan menemukan dokumen yang relevan terlebih dahulu sebelum menyusun jawaban akhir. Lewis et al. (2020) menjelaskan bahwa RAG bekerja dengan menggabungkan dua komponen utama yang beroperasi secara berurutan, yaitu proses pengambilan informasi (*retrieval*) dan proses pembentukan teks (*generation*).

Komponen pertama dalam alur kerja ini adalah *Retriever*. Komponen ini bertugas untuk mengatasi masalah pencarian data yang sangat besar agar sistem tetap efisien dalam menemukan jawaban. *Retriever* bekerja pada sebuah pangkalan data khusus yang disebut *vector store*. Agar sistem dapat memahami maksud pengguna secara mendalam, *Retriever* mengubah setiap pertanyaan menjadi deret angka matematis yang disebut *embedding vector*. Melalui cara ini, sistem tidak lagi sekadar mencocokkan kata kunci yang sama, tetapi mengukur kemiripan makna atau semantik antara pertanyaan pengguna dengan potongan-potongan teks (*chunks*) dari dokumen sumber yang sudah disimpan sebelumnya. Hasil dari proses ini adalah sekumpulan informasi yang paling sesuai dengan konteks kebutuhan pengguna.

Setelah informasi yang tepat berhasil ditemukan, komponen kedua yaitu *Generator* mengambil peran. *Generator* adalah model LLM yang berfungsi mengolah data mentah menjadi jawaban yang mudah dipahami. Pada tahap ini, LLM menerima pertanyaan asli dari pengguna bersama dengan potongan teks

relevan yang dikirimkan oleh *Retriever* sebagai konteks tambahan di dalam perintahnya (*prompt*). Keberadaan konteks ini sangat penting karena memberikan landasan fakta bagi LLM sebelum mulai menuliskan respons. Dengan demikian, LLM tidak lagi memberikan jawaban dari ingatan internalnya yang terbatas, melainkan merangkum informasi dari dokumen resmi yang sudah disediakan di depannya.

Penerapan RAG dalam penelitian ini memiliki peran yang sangat penting, terutama untuk menangani pertanyaan yang bersifat aturan pasti atau normatif dalam konteks fasilitas hunian. Pertanyaan mengenai prosedur denda atau batas waktu kunjungan memerlukan jawaban tunggal yang tepat sesuai dengan isi *Handbook*. Jika sistem hanya mengandalkan LLM standar tanpa RAG, model tersebut akan cenderung menjawab berdasarkan pola umum dari data internet yang belum tentu sesuai dengan aturan khusus asrama. Hal ini berisiko menciptakan konflik antara penghuni dan manajemen akibat pemberian informasi yang keliru.

Gao et al. (2023) dalam kajian mereka menyatakan bahwa integrasi mekanisme pencarian informasi adalah strategi terbaik untuk memastikan luaran LLM tetap berpijak pada sumber kebenaran yang terverifikasi (*grounding*). Strategi ini secara sistematis menghilangkan ruang bagi model untuk mengarang jawaban pada bidang pengetahuan yang sudah terdefinisi dengan jelas. Dalam sistem yang dikembangkan pada penelitian ini, seluruh isi buku panduan diubah menjadi format vektor agar agen yang bertugas dapat selalu melewati jalur RAG. Pendekatan tersebut memastikan bahwa setiap jawaban yang diberikan sistem kepada penghuni dapat ditelusuri kembali sumbernya ke pasal dan halaman tertentu dalam dokumen resmi.

2.8. Otomatisasi Triase Keluhan Fasilitas Hunian

Triase keluhan fasilitas merupakan proses sistematis untuk menilai, mengelompokkan, dan menetapkan laporan kerusakan atau permasalahan infrastruktur berdasarkan jenis, lokasi, serta tingkat urgensinya. Proses ini bertujuan untuk mengarahkan laporan tersebut kepada departemen atau personel yang tepat secara efektif. Di lingkungan fasilitas hunian, triase berfungsi sebagai filter utama yang memastikan bahwa setiap keluhan mendapatkan penanganan sesuai skala

prioritasnya. Tanpa sistem triase yang terstruktur, alokasi sumber daya perawatan fasilitas akan menjadi tidak efisien karena petugas tidak mampu membedakan mana masalah yang memerlukan penanganan segera dan mana yang bersifat rutin.

Pada praktiknya, pengelolaan hunian mahasiswa sering kali menghadapi kendala keterbatasan personel. Satu atau dua staf resepsionis harus melayani ratusan penghuni dari beberapa gedung sekaligus. Kondisi ini diperparah oleh kecenderungan penghuni yang lebih memilih menggunakan kanal percakapan informal seperti WhatsApp dibandingkan aplikasi pelaporan formal. Fenomena ini menciptakan hambatan operasional berupa *format gap*, yaitu kesenjangan antara bahasa sehari-hari yang digunakan penghuni dengan format data terstruktur yang dibutuhkan oleh sistem manajemen. Staf harus bekerja ekstra untuk menerjemahkan pesan teks yang tidak beraturan menjadi kategori data yang baku, seperti menentukan jenis kerusakan, lokasi, dan departemen teknis terkait. Proses penerjemahan manual ini memakan waktu produktif yang cukup besar dan meningkatkan risiko kesalahan input.

Otomatisasi menggunakan *Large Language Model* (LLM) hadir untuk mengatasi kendala tersebut dengan cara mengubah teks tidak terstruktur menjadi instruksi yang dapat dijalankan oleh mesin. Zhong et al. (2024) menjelaskan bahwa arsitektur *Multi-Agent System* (MAS) mampu mengklasifikasikan dan mengarahkan data dari berbagai sumber yang sangat beragam menjadi informasi yang teratur. Prinsip ini sangat relevan jika diterapkan pada triase keluhan fasilitas, di mana sistem dapat secara otomatis membaca pesan keluhan mahasiswa dan menentukan kategorinya tanpa campur tangan manusia. Selain itu, penelitian terbaru oleh Yang et al. (2026) mengenai sistem LIVA menunjukkan bahwa penggunaan agen-agen dengan spesialisasi tertentu dapat menganalisis catatan sistem yang berantakan dan mengubahnya menjadi kategori yang siap ditindaklanjuti. Dalam konteks penelitian ini, teknologi tersebut memungkinkan sistem untuk membedakan antara keluhan darurat dan pertanyaan umum secara instan.

Keunggulan utama dari sistem yang dikembangkan dalam penelitian ini terletak pada integrasi dokumen panduan resmi atau *handbook* hunian. Dokumen ini menjadi basis pengetahuan bagi agen kecerdasan buatan melalui teknik

Retrieval-Augmented Generation (RAG). Dengan metode ini, agen khusus dapat memberikan jawaban yang akurat terkait peraturan, hak, kewajiban, dan prosedur hunian tanpa risiko memberikan informasi palsu atau halusinasi. Sistem tidak hanya berfungsi sebagai alat pengarah keluhan (*routing*), tetapi juga sebagai pusat informasi bagi penghuni. Kombinasi antara triase otomatis yang menggunakan fitur *function calling* dan kemampuan menjawab berdasarkan dokumen resmi inilah yang menjadi inti dari efisiensi sistem otomatisasi pada fasilitas hunian.

2.9. Tinjauan Penelitian Terdahulu

Tinjauan penelitian terdahulu merupakan sebuah proses evaluasi sistematis terhadap karya ilmiah yang telah diterbitkan sebelumnya untuk memetakan perkembangan keilmuan pada bidang tertentu. Proses ini bertujuan untuk memahami penemuan peneliti lain, metode yang mereka gunakan, serta letak kekurangan dari temuan tersebut. Dalam konteks tesis ini, tinjauan literatur berfungsi untuk memetakan perkembangan arsitektur *Multi-Agent System* (MAS) yang berbasis *Large Language Model* (LLM) dan kemampuan *Function Calling*. Pemetaan ini membantu peneliti menemukan celah ilmiah yang belum terselesaikan dan memosisikan penelitian ini sebagai solusi atas permasalahan tersebut.

Perkembangan kecerdasan buatan dalam beberapa tahun terakhir telah melahirkan literatur yang sangat luas mengenai arsitektur MAS dan kapabilitas *Function Calling*. MAS merupakan sebuah sistem yang terdiri dari beberapa agen cerdas. Agen-agen ini saling berinteraksi untuk mencapai tujuan yang sulit diselesaikan oleh satu agen saja. Sementara itu, *Function Calling* adalah fitur yang memungkinkan model bahasa untuk menghubungkan teks yang ia hasilkan dengan perintah teknis pada sistem luar. Kemampuan ini sangat penting karena ia mengubah peran LLM dari sekadar pengolah kata pasif menjadi sistem yang mampu mengambil tindakan nyata. Peneliti perlu memahami perkembangan terbaru dari kedua teknologi ini untuk melihat pilihan arsitektur, cara pengukuran kinerja, dan tempat pengujian teknologi tersebut.

Penelitian-penelitian terbaru menunjukkan bahwa penerapan MAS dan *Function Calling* saat ini lebih banyak berfokus pada bidang rekayasa teknis. Sebagai contoh, teknologi ini digunakan untuk mengatur perangkat pada ekosistem

Internet of Things atau IoT (Rivkin et al., 2025; Zhong et al., 2024). Peneliti lain juga menerapkan arsitektur serupa untuk mengoptimalkan jaringan telekomunikasi generasi keenam (Jiang et al., 2024). Selain itu, sistem koordinasi agen cerdas ini banyak diuji pada aplikasi robotik dan pabrik otomatis (Sun et al., 2025). Bidang-bidang tersebut memiliki lingkungan yang teratur, data yang rapi, dan ukuran keberhasilan yang jelas. Kondisi tersebut menjadikannya tempat pengujian yang sangat baik bagi teknologi baru. Tabel 2.1 berikut membedah sepuluh literatur utama tersebut secara terstruktur untuk mengevaluasi metodologi dan parameter evaluasinya. Pembedahan pada tabel ini sekaligus membuktikan secara sistematis bahwa penerapan arsitektur tersebut pada domain otomatisasi layanan fasilitas hunian masih menjadi celah penelitian yang belum tersentuh.

Tabel 2.2 Tabel Komparasi Penelitian Terdahulu

No	Judul Penelitian (Penulis dan Tahun)	Metode / Arsitektur	Domain / Fokus	Temuan Utama	Keterbatasan / Gap
1	AIoT Smart Home via Autonomous LLM Agents (Rivkin et al., 2025)	Single LLM Agent + Function Calling untuk eksekusi perangkat IoT	Smart Home / Otomasi Perangkat	LLM otonom mampu mengorkestrasi perangkat rumah pintar melalui perintah bahasa natural dengan akurasi eksekusi tinggi	Tidak membandingkan MAS vs. Single Agent, tidak mengevaluasi skenario multi-user dengan volume keluhan tinggi
2	CASIT: Collective Intelligent Agent System for Internet of Things (Zhong et al., 2024)	MAS kolektif untuk triase dan routing data sensor IoT	IoT / Triase Data Sensor	MAS kolektif meningkatkan akurasi triase dan mengurangi latensi routing dibanding pendekatan agen tunggal	Terbatas pada data IoT numerik, tidak menangani input bahasa natural dan tidak mengintegrasikan basis pengetahuan dokumen
3	Large Language Model Enhanced Multi-Agent Systems for 6G Communications (Jiang et al., 2024)	MAS berbasis LLM untuk pengambilan keputusan jaringan telekomunikasi 6G	Telekomunikasi / Pengambilan Keputusan Adaptif	MAS-LLM mengungguli pendekatan konvensional dalam adaptasi alokasi sumber daya jaringan secara real-time	Domain spesifik telekomunikasi, tidak dapat langsung ditransfer ke domain manajemen layanan dan keluhan

No	Judul Penelitian (Penulis dan Tahun)	Metode / Arsitektur	Domain / Fokus	Temuan Utama	Keterbatasan / Gap
4	LLM-Based Multi-Agent Decision-Making: Challenges and Future Directions (Sun et al., 2025)	Kerangka evaluasi dan analisis koordinasi MAS-LLM	Evaluasi Koordinasi dan Komunikasi MAS	Mengidentifikasi context window overflow dan hallucination sebagai tantangan utama Single Agent; koordinasi MAS terbukti memitigasi keduanya	Bersifat survei teoritis, tidak menyediakan implementasi nyata pada kasus penggunaan layanan pelanggan
5	LLM-Based Agents for Tool Learning: A Survey (Xu et al., 2025)	Survei komprehensif arsitektur Function Calling dan API Invocation pada LLM	Function Calling / Tool Use	Mengklasifikasikan dan menganalisis berbagai strategi tool learning; menyimpulkan Function Calling sebagai kapabilitas kunci agen LLM	Tidak mengevaluasi Function Calling dalam konteks triase keluhan atau integrasi dengan sistem manajemen fasilitas
6	A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges (Li et al., 2024)	Survei menyeluruh infrastruktur, workflow, dan tantangan MAS-LLM	Infrastruktur dan Koordinasi MAS	Memetakan lanskap MAS-LLM secara komprehensif; mengidentifikasi gap dalam skalabilitas dan keandalan koordinasi	Tidak membahas integrasi MAS dengan sistem legacy (misalnya: sistem .NET atau API enterprise) untuk otomasi administratif

No	Judul Penelitian (Penulis dan Tahun)	Metode / Arsitektur	Domain / Fokus	Temuan Utama	Keterbatasan / Gap
7	LIVA: A Multi-Agent LLM-Assisted System for IoT Vulnerability Analysis (Yang et al., 2026)	MAS dengan agen terspesialisasi untuk analisis log dan triase celah keamanan IoT	Keamanan IoT / Triase Log Sistem	MAS dengan spesialisasi agen mampu menganalisis dan mengklasifikasikan log kerentanan secara lebih akurat dan efisien dibanding pendekatan agen tunggal	Terbatas pada domain keamanan siber, tidak diterapkan untuk triase keluhan layanan berbasis bahasa natural dari pengguna akhir
8	MAMoE: A Multi-Agent Mixture-of-Experts Framework for LLM-Assisted 3D Object Reconstruction (Liang et al., 2026)	Framework MAS Mixture-of-Experts (MoE) dengan routing tugas ke agen spesialis	Rekonstruksi 3D / Spesialisasi Agen	Routing dinamis ke agen spesialis dalam framework MoE meningkatkan kualitas output secara signifikan dibanding agen generalis tunggal	Domain sangat spesifik (3D rendering), prinsip spesialisasi agen belum divalidasi untuk domain triase keluhan administratif
9	Strategic Learning Under Linguistic and Contextual Constraints: A Theoretical Framework for LLM-Based Multi-Agent Coordination (Yoon, 2025)	Kerangka teori koordinasi MAS di bawah keterbatasan konteks bahasa	Keterbatasan Context Window pada MAS vs. Single Agent	Membuktikan secara formal bahwa keterbatasan context window berdampak lebih parah pada Single Agent dibanding MAS yang terdistribusi	Pendekatan teoritis-matematis, membutuhkan validasi empiris pada kasus penggunaan nyata dengan domain pengetahuan spesifik

No	Judul Penelitian (Penulis dan Tahun)	Metode / Arsitektur	Domain / Fokus	Temuan Utama	Keterbatasan / Gap
10	LLM-Driven Stationarity-Aware Expert Demonstrations for Multi-Agent Reinforcement Learning in Mobile Systems (Duan et al., 2026)	Agen LLM sebagai demonstrator instruksi untuk sistem Multi-Agent Reinforcement Learning	Function Calling / Demonstrasi Instruksi Agen	LLM mampu menghasilkan demonstrasi instruksi berkualitas tinggi yang mempercepat konvergensi pembelajaran multi-agen	Terbatas pada sistem mobile, belum dieksplorasi untuk skenario pelayanan keluhan berbasis bahasa natural dan integrasi API enterprise
11	Scaling Instruction-Finetuned Language Models (Chung et al., 2024)	Scaling laws pada instruction fine-tuning (SFT) untuk berbagai tugas dan model	Instruction Following / SFT	SFT berskala mampu meningkatkan generalisasi LLM secara dramatis; model dengan SFT menghasilkan instruction following jauh lebih baik	Tidak mengevaluasi SFT dalam konteks cloud-based model adaptation untuk sistem triase keluhan dengan basis pengetahuan domain spesifik
12	Self-Instruct: Aligning Language Models with Self-Generated Instructions (Wang et al., 2023)	Generasi data instruksi secara mandiri untuk fine-tuning LLM	Data SFT / Instruction Alignment	Model yang di-fine-tune dengan instruksi berkualitas tinggi termasuk yang dihasilkan sendiri menunjukkan kemampuan instruction following yang sangat baik	Tidak mengevaluasi efektivitas metode pada bahasa Indonesia informal atau konteks domain fasilitas hunian mahasiswa

Analisis menyeluruh terhadap Tabel 2.1 membuktikan sebuah pola yang kuat mengenai keunggulan arsitektur MAS. Sepuluh penelitian tersebut secara kolektif menunjukkan bahwa pembagian tugas kepada beberapa agen cerdas jauh lebih efektif daripada mengandalkan agen tunggal. Keunggulan ini sangat menonjol untuk tugas-tugas yang memerlukan koordinasi dan pemrosesan multi-domain. Keberhasilan ini telah divalidasi di berbagai bidang, mulai dari jaringan komunikasi Jiang et al. (2024) hingga keamanan siber Yang et al. (2026). Selain itu, mekanisme *Function Calling* terkonfirmasi sebagai bagian yang sangat kritis. Fitur ini memungkinkan agen untuk berinteraksi dengan dunia luar secara nyata dan terukur (Rivkin et al., 2025; Xu et al., 2025).

Meskipun fondasi teoritisnya sudah sangat kuat, terdapat satu kekurangan besar yang terlihat jelas pada kolom keterbatasan di Tabel 2.1. Sebagian besar penelitian belum menguji performa MAS secara langsung melawan agen tunggal dalam skenario otomatisasi triase keluhan di fasilitas hunian. Bidang ini memiliki tantangan unik karena sangat bergantung pada masukan teks bahasa natural yang beragam dan tidak terstruktur dari pengguna non-teknis. Belum ada penelitian yang menggabungkan tiga komponen utama secara simultan. Ketiga komponen tersebut meliputi penggunaan buku panduan resmi sebagai penjaga ketepatan informasi, fitur *Function Calling* untuk membuat perintah terstruktur, dan integrasi dengan antarmuka pemrograman aplikasi (API) dari sistem lama berbasis *.NET*. Padahal, ketiga komponen ini merepresentasikan kondisi teknis yang dihadapi oleh pengelola hunian mahasiswa.

Kekosongan ini merepresentasikan sebuah jarak antara bukti penelitian teoritis dengan kebutuhan operasional praktisi di lapangan. Pengelola fasilitas sering kali menghadapi masalah keterbatasan staf, di mana satu atau dua resepsionis harus melayani keluhan dari ratusan penghuni di berbagai gedung. Penelitian ini hadir untuk menjembatani jarak tersebut. Peneliti merancang eksperimen perbandingan yang terkontrol antara arsitektur MAS dan agen tunggal dalam menangani keluhan fasilitas secara nyata. Evaluasi dilakukan menggunakan metrik operasional yang relevan, seperti ketepatan klasifikasi, tingkat halusinasi (*hallucination rate*), latensi, serta waktu rata-rata pemulihan masalah atau *Mean Time To Repair* (MTTR). Fokus pada penyelesaian masalah praktis melalui

integrasi *webhook* dan API inilah yang menjadi fondasi utama kebaruan ilmiah dalam penelitian ini.

BAB III

METODOLOGI PENELITIAN

3.1. Kerangka Berpikir

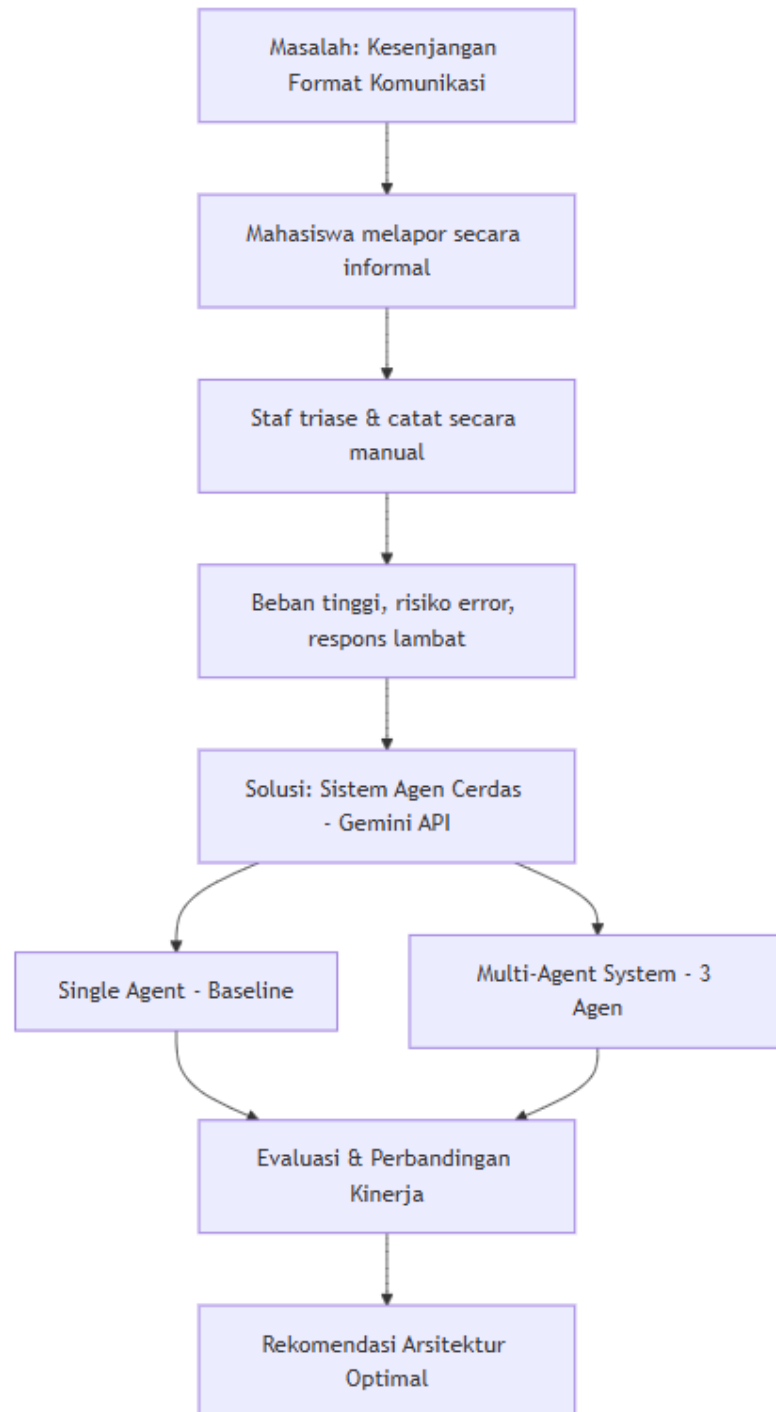
Penelitian ini bermula dari sebuah masalah nyata yang belum terpecahkan. Masalah tersebut berwujud sebuah kesenjangan yang cukup besar antara cara mahasiswa berkomunikasi dengan cara sistem manajemen fasilitas bekerja. Mahasiswa yang ingin melaporkan kerusakan di kamarnya cenderung memilih jalur yang paling mudah dan akrab dengan mengirim pesan singkat melalui WhatsApp atau berbicara langsung kepada staf di resepsionis. Cara ini terasa alami bagi mereka, tetapi justru menciptakan beban kerja yang besar bagi staf resepsionis yang harus menerjemahkan pesan informal tersebut menjadi data terstruktur yang bisa dimasukkan ke dalam sistem pelaporan digital.

Kondisi ini diperparah oleh keterbatasan jumlah staf. Di fasilitas hunian Kampus XYZ, satu atau dua resepsionis harus melayani keluhan dari ribuan mahasiswa yang menghuni empat gedung sekaligus. Ketika jam sibuk tiba, antrian keluhan menumpuk, risiko kesalahan pencatatan meningkat, dan waktu penanganan menjadi semakin lama. Penelitian ini hadir untuk menjawab tantangan tersebut melalui sebuah sistem kecerdasan buatan yang mampu menjembatani kesenjangan format itu secara otomatis.

Jembatan yang dirancang dalam penelitian ini adalah sebuah sistem agen cerdas berbasis *Large Language Model* (LLM) dengan menggunakan layanan Gemini API dari Google Cloud. Sistem ini dirancang dalam dua varian arsitektur yang akan dibandingkan secara langsung. Varian pertama adalah arsitektur *Single Agent*, di mana satu model tunggal bertanggung jawab atas seluruh proses, mulai dari menerima pesan mahasiswa, menjawab pertanyaan berdasarkan panduan penghuni (*Handbook*), hingga membuat tiket laporan di sistem manajemen. Varian kedua adalah arsitektur *Multi-Agent System* (MAS), di mana tanggung jawab tersebut dibagi kepada tiga agen yang masing-masing memiliki peran khusus dan bekerja secara terkoordinasi.

Kedua arsitektur ini akan diuji menggunakan data skenario keluhan yang bersumber dari riwayat fasilitas hunian Kampus XYZ. Hasilnya diukur

menggunakan serangkaian metrik yang mencakup ketepatan klasifikasi, kebenaran informasi, efisiensi komputasi, dan kecepatan penanganan keluhan. Seluruh alur pemikiran penelitian ini digambarkan secara sistematis pada Gambar 3.1.



Gambar 3.1 Alur Pemikiran Penelitian

3.2. Tahapan Penelitian

Penelitian ini akan dijalankan melalui lima tahapan yang tersusun secara berurutan. Setiap tahapan dirancang untuk menghasilkan sebuah keluaran yang menjadi fondasi bagi tahapan berikutnya, sehingga seluruh proses berjalan dengan terarah dan terukur.

Tahap pertama adalah studi pendahuluan. Pada tahap ini, peneliti akan melakukan kajian mendalam terhadap kondisi operasional fasilitas hunian Kampus XYZ. Kajian ini mencakup memahami alur kerja triase yang berjalan saat ini, analisis terhadap dokumen *Handbook* sebagai basis pengetahuan sistem, serta studi terhadap spesifikasi teknis antarmuka pemrograman berbasis .NET yang telah tersedia di lingkungan kampus. Tahap ini juga mencakup tinjauan literatur sebagaimana yang telah dirangkum dalam Bab II, yang memberikan landasan teoritis bagi pilihan arsitektur dan metode evaluasi dalam penelitian ini.

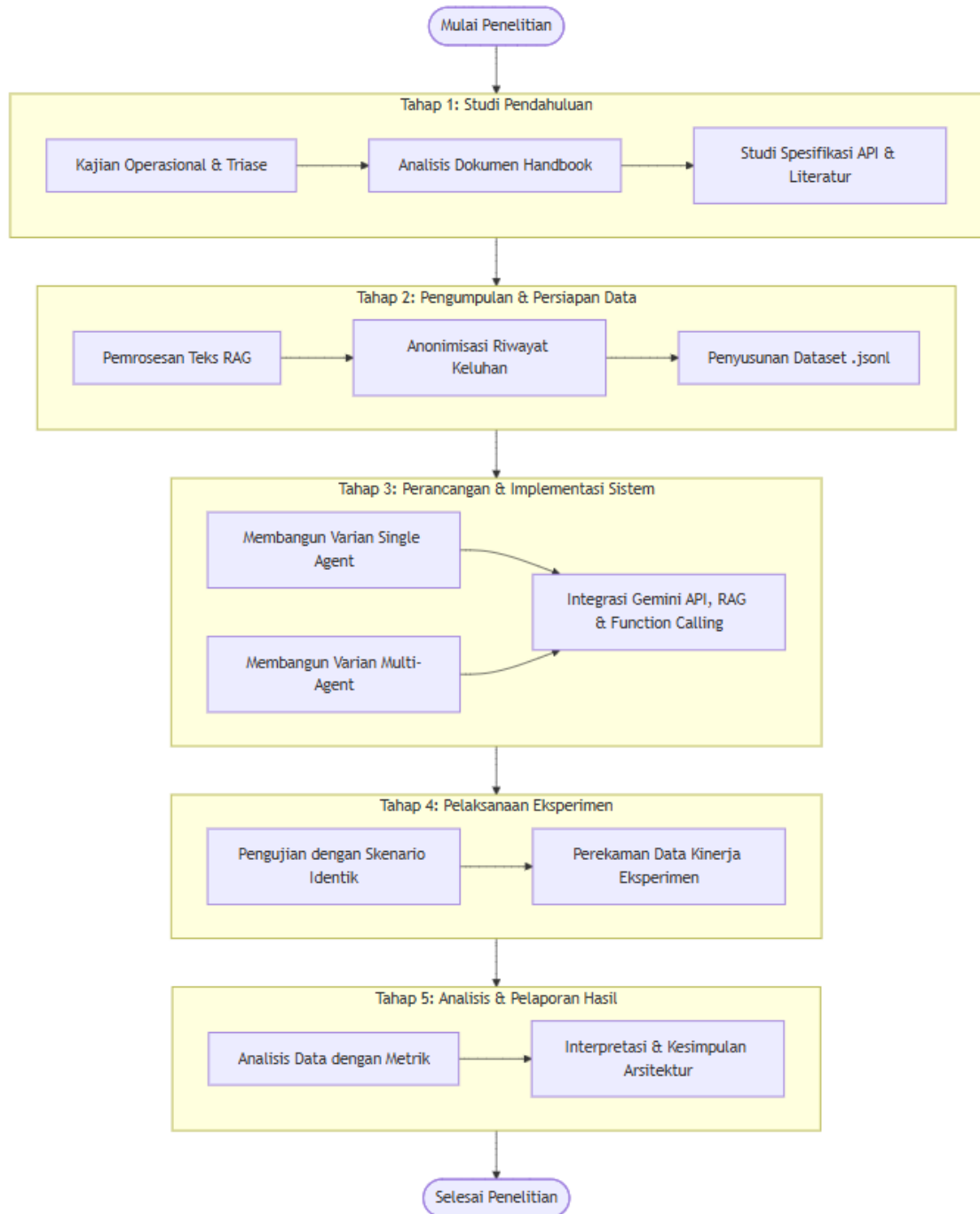
Tahap kedua adalah pengumpulan dan persiapan data. Peneliti akan memproses *Handbook* menjadi potongan-potongan teks yang siap diindeks untuk keperluan *Retrieval-Augmented Generation* (RAG). Selain itu, peneliti akan menyusun dataset skenario keluhan dalam format .jsonl untuk keperluan penyesuaian instruksi (*instruction fine-tuning*) model. Seluruh data riwayat keluhan yang digunakan akan melalui proses anonimisasi terlebih dahulu untuk menjaga privasi penghuni.

Tahap ketiga adalah perancangan dan implementasi sistem. Pada tahap ini, peneliti akan membangun dua varian sistem secara paralel, yaitu sistem *Single Agent* dan sistem *Multi-Agent System* (MAS). Kedua sistem menggunakan Gemini API sebagai model bahasa utama dan dilengkapi dengan mekanisme RAG serta kemampuan *function calling* untuk berinteraksi dengan API manajemen fasilitas.

Tahap keempat adalah pelaksanaan eksperimen. Peneliti akan menjalankan kedua sistem menggunakan dataset pengujian yang telah disiapkan. Setiap sistem akan dihadapkan pada skenario keluhan yang identik, sehingga perbandingan kinerja yang dihasilkan bersifat adil dan terstandarisasi.

Tahap kelima adalah analisis dan pelaporan hasil. Seluruh data kinerja yang terkumpul dari eksperimen akan dianalisis menggunakan metrik yang telah ditetapkan. Hasil analisis kemudian diinterpretasikan untuk menarik kesimpulan

mengenai arsitektur mana yang lebih optimal untuk skenario triase keluhan di fasilitas hunian mahasiswa.



Gambar 3.2 Alur Tahapan Penelitian

3.3. Pengumpulan & Persiapan Data

Data merupakan fondasi utama yang menentukan kualitas sistem kecerdasan buatan. Tanpa data yang tepat dan representatif, sistem tidak akan mampu memahami konteks keluhan mahasiswa maupun menjawab pertanyaan mereka secara akurat. Penelitian ini merancang proses pengumpulan data yang terbagi menjadi dua jalur utama sesuai dengan kebutuhan teknis sistem yang akan dibangun.

Tahapan pertama adalah persiapan data untuk komponen RAG. Sistem RAG membutuhkan sebuah basis pengetahuan yang dapat dicari secara semantik. Dalam penelitian ini, basis pengetahuan tersebut bersumber dari *Handbook* resmi fasilitas hunian Kampus XYZ tahun 2025-2026, yang memuat seluruh aturan dan prosedur yang berlaku, mulai dari tata cara *check-in* dan *check-out*, kode etik penghuni, jadwal dan ketentuan penggunaan fasilitas umum, hingga daftar biaya penggantian kerusakan. Dokumen tersebut akan dipecah menjadi potongan-potongan teks berukuran sekitar 300 hingga 500 kata per potongan menggunakan metode pemisahan berbasis paragraf. Setiap potongan kemudian diubah menjadi vektor numerik menggunakan model *embedding* dari *Google Cloud*, lalu disimpan ke dalam sebuah basis data vektor. Proses inilah yang memungkinkan sistem untuk mencari bagian *Handbook* yang paling relevan setiap kali ada pertanyaan dari mahasiswa.

Tahapan kedua adalah persiapan data untuk proses SFT (*Supervised Fine-Tuning*). Data ini bertujuan untuk menyesuaikan perilaku model *Gemini API Model* agar lebih mahir menangani konteks percakapan keluhan fasilitas hunian dalam bahasa Indonesia, termasuk penggunaan bahasa sehari-hari yang tidak baku, kosakata gaul, serta ekspresi informal yang lazim digunakan oleh mahasiswa. Mengacu pada penelitian Wang et al. (2023) mengenai *Self-Instruct*, kualitas dan keragaman data instruksi menjadi faktor penentu utama keberhasilan proses SFT. Data SFT dalam penelitian ini disusun dalam format *.jsonl*, di mana setiap baris berisi satu pasangan percakapan yang terdiri atas pesan pengguna dan respons ideal yang diharapkan dari sistem. Skenario yang dicakup meliputi laporan kerusakan fasilitas, pertanyaan aturan hunian, permintaan informasi jadwal, hingga respons

situasi darurat, dengan total rencana data minimal 500 pasangan skenario untuk memastikan kemampuan generalisasi model.

Proses SFT tersebut dijalankan melalui lingkungan Vertex AI dari Google Cloud Platform, mengikuti paradigma *cloud-based model adaptation* yang telah diuraikan pada Bab II. Dataset yang telah disiapkan diunggah ke Cloud Storage dan dikonfigurasi sebagai data pelatihan untuk sesi SFT pada *Gemini API Model* yang menjadi fondasi sistem.

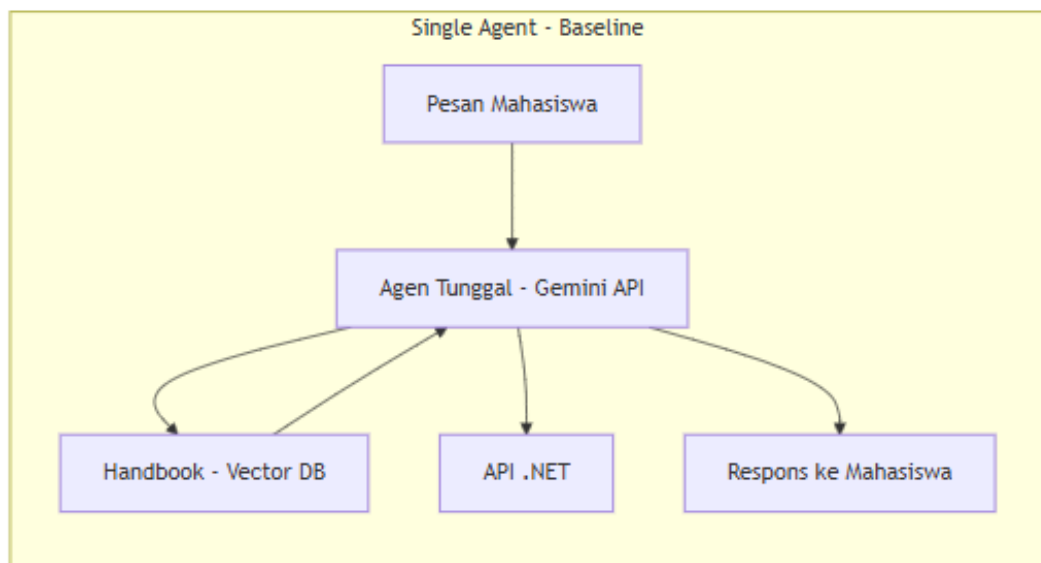
Penggunaan data dalam penelitian ini telah disetujui oleh pihak manajemen fasilitas, sementara surat persetujuan resmi sedang dalam tahap proses administratif. Untuk menjaga kerahasiaan dan privasi penghuni, peneliti akan melakukan proses anonimisasi terhadap seluruh data riwayat keluhan. Proses anonimisasi tersebut mencakup penghapusan data sensitif yang berpotensi mengidentifikasi individu, seperti nama penghuni, nomor kamar, dan nomor identifikasi mahasiswa, sebelum data tersebut digunakan untuk pengujian sistem.

3.4. Skenario *Single Agent* (Sistem Pemandangan)

Sebelum membangun sistem yang kompleks, peneliti perlu menetapkan titik acuan kinerja yang jelas. Titik acuan ini dikenal sebagai *baseline*. Dalam penelitian ini, *baseline* ditetapkan melalui sebuah arsitektur *Single Agent* yang sederhana namun lengkap secara fungsional.

Single Agent adalah sebuah model Gemini tunggal yang bertanggung jawab penuh atas seluruh alur penanganan keluhan. Ketika seorang mahasiswa mengirimkan pesan, agen tunggal ini harus mampu menyelesaikan serangkaian tugas sekaligus dalam satu sesi pemrosesan. Agen tersebut harus memutuskan apakah pesan yang masuk merupakan keluhan fasilitas yang perlu dibuatkan tiket, pertanyaan umum yang perlu dijawab berdasarkan *Handbook*, atau permintaan yang di luar kapasitasnya. Jika merupakan keluhan, agen harus mengklasifikasikan jenis kerusakannya, menentukan departemen tujuan yang tepat, mengekstraksi informasi penting dari pesan, dan menghasilkan data JSON yang valid untuk dikirimkan ke API manajemen. Jika merupakan pertanyaan, agen harus mencari informasi yang relevan dari *Handbook* melalui mekanisme RAG dan memberikan jawaban yang akurat.

Seluruh beban tugas ini ditanggung oleh satu *context window* yang sama. Kondisi ini menciptakan potensi masalah yang telah diidentifikasi dalam tinjauan literatur, yaitu penurunan akurasi dan peningkatan risiko halusinasi seiring bertambahnya kompleksitas percakapan. Skenario pengujian *Single Agent* akan dijalankan dengan konfigurasi yang identik dengan yang digunakan pada sistem MAS, termasuk versi model Gemini, dan dataset pengujian. Dengan demikian, perbedaan kinerja yang terukur nantinya dapat benar-benar dikaitkan dengan perbedaan arsitektur, bukan perbedaan kondisi pengujian.



Gambar 3.3 Alur Single Agent

3.5. Agen 1: Penerima Pesan & RAG

Dalam arsitektur *Multi-Agent System* yang dirancang dalam penelitian ini, tidak semua agen perlu mengetahui segalanya. Prinsip inilah yang menjadi kekuatan utama MAS, setiap agen cukup ahli dalam satu bidang saja, sehingga kinerjanya lebih tajam dan lebih dapat diandalkan.

Agen pertama berperan sebagai pintu masuk utama sistem. Setiap pesan yang dikirimkan oleh mahasiswa akan pertama kali diterima oleh agen ini. Tugasnya yang paling mendasar adalah memahami maksud dari pesan tersebut dan menentukan tindakan awal yang tepat. Jika pesan berisi pertanyaan tentang aturan atau prosedur hunian, agen ini akan langsung mengaktifkan mekanisme RAG untuk

mencari jawaban yang relevan dari *Handbook*. Cara kerjanya adalah dengan mengubah pertanyaan mahasiswa menjadi sebuah vektor pencarian, lalu membandingkannya dengan vektor-vektor potongan *Handbook* yang tersimpan di dalam basis data vektor. Potongan *Handbook* yang paling mirip secara makna akan diambil dan disertakan sebagai konteks tambahan dalam perintah (prompt) yang dikirimkan kepada model Gemini. Dengan cara ini, model tidak mengarang jawaban dari ingatannya sendiri, melainkan merangkum informasi dari dokumen resmi yang sudah terjamin kebenarannya.

Jika pesan yang masuk teridentifikasi sebagai laporan keluhan dan bukan pertanyaan, maka Agen 1 tidak akan mencoba menanganinya sendiri. Sebaliknya, agen ini meneruskan pesan tersebut beserta konteks percakapan yang relevan kepada Agen 2 yang memiliki keahlian khusus dalam triase dan klasifikasi. Pembagian kerja yang tegas ini memastikan context window Agen 1 tidak dipenuhi oleh informasi yang tidak perlu, sehingga kemampuannya dalam menjawab pertanyaan berbasis *Handbook* tetap terjaga dengan baik.

3.6. Agen 2: Triase & Klasifikasi

Setelah Agen 1 mengidentifikasi bahwa sebuah pesan merupakan laporan keluhan, giliran Agen 2 untuk bekerja. Agen ini adalah spesialis triase. Triase adalah proses memilah dan memprioritaskan suatu masalah sebelum menentukan langkah penanganannya yang tepat.

Agen 2 menerima pesan keluhan yang telah diidentifikasi oleh Agen 1, kemudian menganalisisnya secara mendalam. Proses analisis ini mencakup beberapa dimensi sekaligus. Dimensi pertama adalah kategori kerusakan: apakah ini masalah kelistrikan, pipa air, perabot, pendingin ruangan, akses internet, atau kategori lainnya yang tercantum dalam sistem manajemen fasilitas? Dimensi kedua adalah lokasi kejadian: di kamar mana, di lantai berapa, atau di area fasilitas umum yang mana? Dimensi ketiga adalah tingkat urgensi, apakah ini darurat yang membutuhkan penanganan segera, atau bisa menunggu jadwal pemeliharaan rutin?

Berdasarkan hasil analisis tersebut, Agen 2 akan melakukan pemanggilan fungsi (*function calling*) untuk mengakses sebuah alat bantu yang telah didefinisikan sebelumnya, yaitu fungsi pengklasifikasi departemen. Fungsi ini

bekerja seperti sebuah direktori yang memetakan setiap kombinasi kategori dan urgensi keluhan kepada departemen teknis yang bertanggung jawab menanganinya, misalnya Tim Estate untuk kerusakan fisik atau Tim IT untuk gangguan jaringan internet. Keluaran dari Agen 2 adalah sebuah objek data yang telah terklasifikasi dengan rapi, siap untuk diserahkan kepada Agen 3 guna diproses lebih lanjut menjadi tiket pelaporan yang resmi.

3.7. Agen 3: Eksekutor Tiket

Agen ketiga adalah ujung dari rantai proses otomatisasi. Jika Agen 1 berperan sebagai penerima dan penjawab, dan Agen 2 berperan sebagai analis dan penilai, maka Agen 3 berperan sebagai pelaksana. Tugasnya adalah mengubah hasil analisis Agen 2 menjadi sebuah tiket pelaporan yang nyata di dalam sistem manajemen fasilitas.

Proses kerja Agen 3 dimulai dengan menerima data terklasifikasi dari Agen 2. Berdasarkan data tersebut, Agen 3 menyusun sebuah objek JSON yang strukturnya mengikuti skema yang ditetapkan oleh API manajemen fasilitas berbasis .NET yang telah tersedia di lingkungan Kampus XYZ. Objek JSON ini memuat seluruh informasi yang diperlukan untuk membuat tiket, seperti kategori keluhan, lokasi, deskripsi masalah, tingkat urgensi, dan departemen tujuan. Contoh struktur JSON yang dihasilkan oleh Agen 3 adalah sebagai berikut.

```
{
  "function": "create_maintenance_ticket",
  "parameters": {
    "category": "Kelistrikan",
    "location": "Lantai 3 - Lorong",
    "description": "Lampu lorong padam",
    "urgency": "Sedang",
    "target_department": "Tim Teknik"
  }
}
```

Tabel 3.1 Contoh Schema JSON

Setelah JSON tersusun dengan benar, Agen 3 mengirimkannya ke *endpoint* API melalui sebuah permintaan HTTP POST. Jika API memberikan respons konfirmasi bahwa tiket berhasil dibuat, Agen 3 meneruskan informasi nomor tiket tersebut kembali ke Agen 1 untuk disampaikan kepada mahasiswa sebagai bukti bahwa laporannya telah diterima dan sedang dalam proses penanganan. Perbandingan lengkap antara alur kerja *Single Agent* dan *alur kerja Multi-Agent System* dalam penelitian ini digambarkan pada Gambar 3.2.

3.8. Orkestrasi dan Integrasi Multi-Agent System

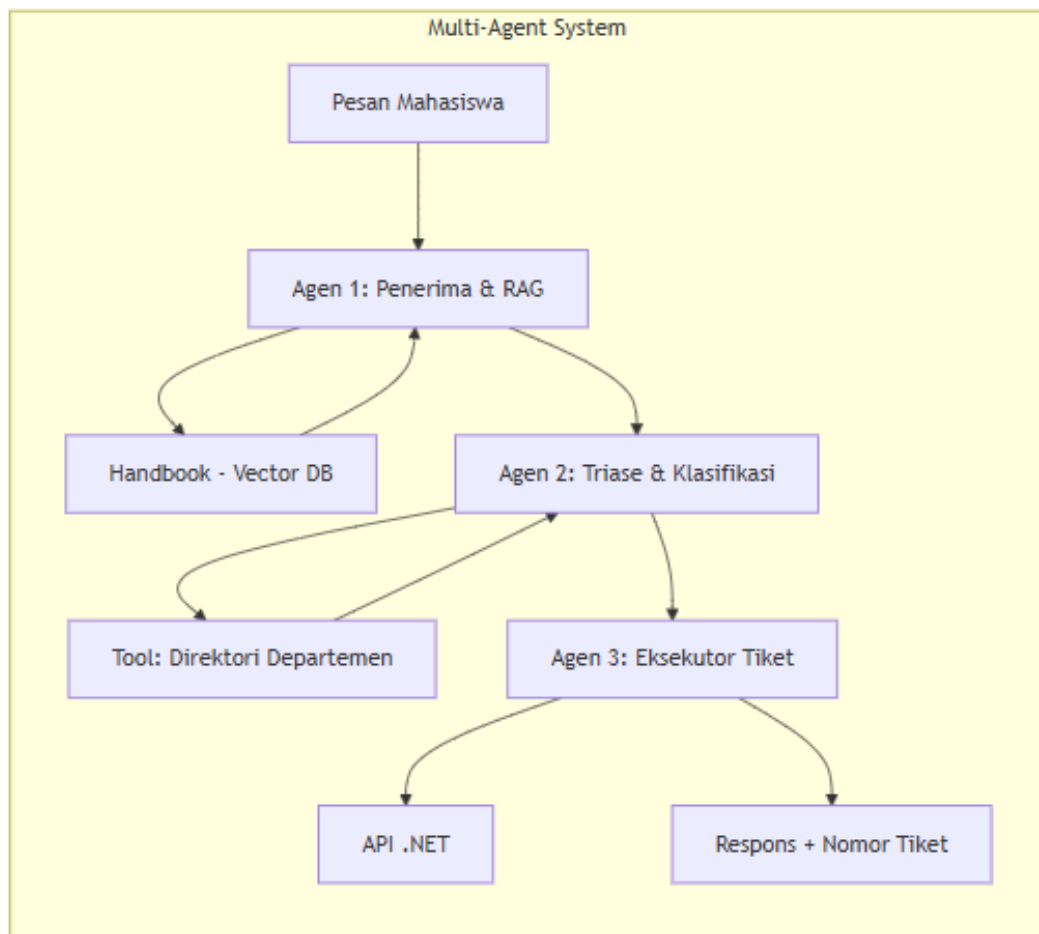
Dalam arsitektur *Multi-Agent System* (MAS), penggabungan kerja antar-agen atau orkestrasi berfungsi sebagai pengatur alur data untuk memastikan setiap komponen bekerja sesuai perannya. Pembagian tugas yang terstruktur ini sangat penting untuk mengatasi kelemahan utama pada model agen tunggal, yaitu keterbatasan memori sistem atau *context window*. Ketika satu agen dipaksa membaca buku panduan, memilah masalah, dan membuat laporan secara bersamaan, memori agen akan cepat penuh dan risiko munculnya jawaban keliru (halusinasi) akan meningkat drastis. Oleh karena itu, sistem ini merangkai Agen 1, Agen 2, dan Agen 3 ke dalam satu alur kerja yang berurutan, sehingga setiap agen hanya memproses informasi yang benar-benar relevan dengan keahliannya.

Proses kerja sama yang saling berkaitan ini dimulai dari Agen 1, yang bertindak sebagai pintu gerbang utama untuk menerima pesan dari mahasiswa. Saat pesan masuk, Agen 1 bertugas memahami apa yang sebenarnya diinginkan oleh pengguna. Jika pesan berupa pertanyaan umum seputar aturan hunian, agen ini akan mencari jawaban di buku panduan menggunakan metode *Retrieval-Augmented Generation* (RAG). Namun, jika pesan tersebut dikenali sebagai laporan kerusakan fasilitas, Agen 1 tidak akan mencoba menyelesaikannya sendiri. Ia akan langsung menghentikan proses mandiri dan segera meneruskan pesan tersebut kepada Agen 2. Langkah pendelegasian ini sengaja dirancang agar kapasitas memori Agen 1 tidak terbebani oleh kerumitan pendataan teknis, sehingga ia tetap responsif dan cepat dalam melayani sesi tanya jawab.

Setelah menerima data laporan dari Agen 1, Agen 2 mengambil alih kemudi sebagai spesialis triase yang membedah masalah secara mendalam. Tugas

utamanya adalah menyaring teks percakapan sehari-hari dari mahasiswa untuk menemukan tiga informasi kunci: jenis kerusakan, lokasi pasti kejadian, dan seberapa mendesak masalah tersebut harus segera ditangani. Berdasarkan ketiga informasi teknis tersebut, Agen 2 menggunakan kemampuan pemanggilan fungsi atau *function calling* untuk mencocokkannya dengan buku direktori departemen teknis. Kemampuan ini memastikan bahwa setiap keluhan diarahkan secara otomatis ke tim yang tepat, misalnya kerusakan lampu ke tim kelistrikan. Hasil pemilahan ini kemudian dibungkus menjadi sebuah data yang teratur dan diserahkan kepada Agen 3.

Pada tahap penutup, Agen 3 bertindak sebagai pihak eksekutor yang menghubungkan kecerdasan buatan dengan sistem manajemen fasilitas yang sebenarnya di kampus. Agen 3 mengambil data rapi dari Agen 2 dan menyusunnya ulang ke dalam format baku *JavaScript Object Notation* (JSON). Format ini dirancang secara spesifik agar dapat dibaca dan diterima oleh kerangka *Application Programming Interface* (API) berbasis .NET milik kampus. Setelah laporan terkirim dan berhasil dicatat sebagai tiket resmi, sistem pusat hunian akan membalas dengan sebuah nomor tiket pelaporan. Nomor tiket ini kemudian dikirim kembali secara berantai kepada Agen 1 untuk disampaikan kepada mahasiswa sebagai bukti nyata bahwa keluhan mereka telah ditangani. Siklus komunikasi dua arah ini tidak hanya menyederhanakan proses pelaporan bagi penghuni, tetapi juga membuktikan efektivitas sistem yang terintegrasi secara utuh.



Gambar 3.4 Alur Multi-Agent System

3.9. Metode Evaluasi

Evaluasi kinerja sistem kecerdasan buatan merupakan tahapan pengukuran terstruktur untuk menilai kemampuan sebuah model dalam mencapai sasaran rancangannya. Pengukuran ini menyediakan acuan perbandingan yang stabil secara matematis. Peneliti membutuhkan standar ukur ini untuk membedakan kualitas antara satu model dengan model lainnya secara objektif. Pada sistem berbasis *Multi-Agent System* (MAS), pengukuran memberikan bukti wujud berupa persentase dan angka ukur teknis. Bukti ini sangat penting untuk memastikan apakah pembagian tugas kepada agen-agen khusus benar-benar memberikan hasil kerja yang lebih baik daripada pemakaian satu agen tunggal. Penelitian ini menggunakan empat kelompok metrik utama untuk menguji kelayakan dan keandalan arsitektur yang peneliti usulkan.

Kelompok pengukuran pertama menilai keandalan teknis sistem dalam menjalankan perintah kerja. Peneliti membaginya menjadi dua metrik turunan, yaitu akurasi klasifikasi dan validitas *function calling*. Akurasi klasifikasi mengukur ketepatan sistem saat mengarahkan teks keluhan pengguna ke departemen teknis yang benar. Peneliti membandingkan keluaran arah dari sistem dengan data acuan kebenaran atau *ground truth* yang telah pakar dari pengelola hunian susun. Peneliti menghitung tingkat akurasi klasifikasi tersebut menggunakan persamaan matematis

$$\text{Akurasi} = \frac{K_{\text{benar}}}{N} \times 100\% \quad (3.1)$$

Di mana:

- K_{benar} : Jumlah keluhan yang berhasil diklasifikasikan ke departemen yang tepat (sesuai *ground truth*).
- N : Total populasi atau jumlah seluruh keluhan yang dievaluasi.

Selanjutnya, metrik validitas *function calling* memastikan bahwa teks instruksi dari *Large Language Model* (LLM) tersusun rapi sehingga dapat dibaca oleh perangkat lunak lain. Kegagalan pada tahap translasi struktur data ini akan menghentikan seluruh proses otomatisasi pada alur kerja berikutnya (Xu et al., 2025). Peneliti merumuskan pengujian validitas ini dengan persamaan

$$\text{Validitas}_{\text{JSON}} = \frac{J_{\text{valid}}}{N_{\text{call}}} \times 100\% \quad (3.2)$$

Di mana:

- J_{valid} : Jumlah keluaran sistem yang berhasil di-parse dan memiliki struktur JSON yang valid sesuai skema API.
- N_{call} : Total seluruh pemanggilan fungsi (*function calling*) atau *request* yang dilakukan oleh sistem.

Kelompok pengukuran kedua menilai mutu dan kebenaran informasi yang sistem hasilkan untuk penghuni. Fokus paling mendasar pada bagian ini adalah perhitungan tingkat halusinasi atau *hallucination rate*. Metrik ini menghitung seberapa sering agen cerdas memberikan jawaban yang melenceng, mengarah bebas, atau bertentangan dengan buku panduan penghuni (*Handbook*). Penjelasan

sistem yang salah sangat berisiko karena dapat memicu perdebatan antara penghuni dan staf pengelola fasilitas. Pembebanan ragam instruksi yang terlalu banyak pada satu memori agen sering menjadi pemicu utama munculnya halusinasi informasi ini (Sun et al., 2025). Oleh karena itu, peneliti perlu menghitung tingkat kesalahan ini secara rinci melalui perbandingan manual. Rumus matematis yang peneliti terapkan adalah

$$HR = \frac{R_{\text{halusinasi}}}{R_{\text{total}}} \times 100\% \quad (3.3)$$

Di mana:

- HR : Tingkat Halusinasi (*Hallucination Rate*).
- $R_{\text{halusinasi}}$: Jumlah respons sistem keliru atau tidak sesuai dengan *Handbook*.
- R_{total} : Total seluruh respons sistem yang diverifikasi secara manual.

Kelompok pengukuran ketiga mengevaluasi efisiensi pemakaian sumber daya komputasi. Peneliti memecah bagian pengujian ini menjadi waktu tunggu (latensi) dan penggunaan token. Waktu tunggu mencatat durasi penyelesaian yang sistem butuhkan sejak pesan masuk diterima hingga balasan akhir keluar. Pengukuran kecepatan ini penting karena arsitektur MAS memerlukan pertukaran data antar agen yang dapat memakan waktu. Peneliti harus memastikan proses koordinasi di belakang layar ini tidak membuat pengguna menunggu terlalu lama (Li et al., 2024). Peneliti menghitung rata-rata waktu tunggu tersebut melalui fungsi

$$\bar{L} = \frac{1}{N} \sum_{i=1}^N (t_{\text{out},i} - t_{\text{in},i}) \quad (3.4)$$

Di mana:

- $\bar{L}$: Rata-rata latensi respons
- N : Total jumlah permintaan (*request*) yang diproses.
- $t_{\text{out},i}$: Waktu (*timestamp*) saat respons ke- i selesai dihasilkan dan dikirim oleh sistem.
- $t_{\text{in},i}$: Waktu (*timestamp*) saat pesan ke- i pertama kali diterima oleh sistem.

Sementara itu, efisiensi token menghitung rata-rata pemakaian karakter kognitif mesin pada setiap transaksi. Penghematan token akan secara langsung memangkas tagihan pemakaian API berbayar serta mengurangi beban memori komputasi. Rumus perhitungannya adalah

$$\bar{T} = \frac{1}{N} \sum_{i=1}^N (T_{\text{input},i} + T_{\text{output},i}) \quad (3.5)$$

Di mana:

- $\bar{T}$: Rata-rata penggunaan total token per transaksi.
- N : Total jumlah transaksi yang dievaluasi.
- $t_{\text{out},i}$: Jumlah token *prompt* (masukan) yang dikonsumsi pada transaksi ke- i .
- $t_{\text{in},i}$: Jumlah token *completion* (keluaran) yang dihasilkan pada transaksi ke- i .

Kelompok pengukuran keempat dan terakhir menilai dampak sistem secara langsung terhadap percepatan operasional harian. Pengukuran ini menggunakan standar mapan dari ranah manajemen layanan teknologi informasi bernama *Mean Time To Resolve* (MTTR). MTTR menghitung waktu rata-rata yang diperlukan untuk menuntaskan masalah secara menyeluruh. Perhitungan dimulai sejak detik pertama keluhan masuk hingga teknisi pengelola menyatakan masalah tersebut tuntas. Kehadiran sistem triase otomatis secara teoretis akan memotong waktu tunggu ketika staf manusia sedang sibuk melayani penghuni lain. Otomatisasi ini memastikan keluhan darurat langsung membunyikan sistem notifikasi di pihak yang tepat (Zhong et al., 2024). Penurunan angka waktu MTTR ini akan menjadi bukti nyata bahwa integrasi LLM berdampak positif terhadap kualitas hidup penghuni kampus. Peneliti mencari nilai percepatan pelayanan ini melalui kalkulasi matematis

$$MTTR = \frac{1}{N} \sum_{i=1}^N (T_{\text{selesai},i} + T_{\text{lapor},i}) \quad (3.6)$$

Di mana:

- $\bar{T}$: Rata-rata penggunaan total token per transaksi.
- N : Total jumlah transaksi yang dievaluasi.

- $t_{out,i}$: Jumlah token *prompt* (masukan) yang dikonsumsi pada transaksi ke-i.
- $t_{in,i}$: Jumlah token *completion* (keluaran) yang dihasilkan pada transaksi ke-i.

3.10. Jadwal Penelitian

Jadwal penelitian disusun sebagai panduan langkah demi langkah agar seluruh proses pembuatan sistem dapat berjalan terarah dan selesai tepat waktu. Penelitian ini dijadwalkan berlangsung selama delapan bulan, terhitung dari bulan Juni 2026 hingga Januari 2027. Waktu delapan bulan ini dinilai cukup untuk menyelesaikan setiap tahapan secara berurutan, mulai dari mempelajari masalah pelaporan di asrama hingga menguji coba kecerdasan buatan secara langsung. Seluruh rencana kegiatan tersebut dirangkum dengan jelas pada Tabel 3.1 berikut.

Tabel 3.2 Rincian Jadwal Kegiatan Penelitian

No	Tahapan Kegiatan	Bulan							
		6	7	8	9	10	11	12	1
1	Studi Pendahuluan & Tinjauan Literatur	✓	✓						
2	Pengumpulan & Persiapan Data		✓	✓					
3	Perancangan & Implementasi Sistem			✓	✓				
4	Pelaksanaan Eksperimen					✓	✓		
5	Analisis Hasil & Penulisan Laporan						✓	✓	✓

Langkah pertama dalam jadwal ini adalah studi pendahuluan yang berlangsung pada bulan Juni hingga Juli. Pada tahap ini, fokus utamanya adalah mengamati langsung bagaimana staf menangani keluhan penghuni saat ini. Bersamaan dengan itu, dilakukan juga pembacaan berbagai jurnal ilmiah terbaru mengenai teknologi *function calling*. Tujuannya sangat sederhana, yaitu memastikan bahwa solusi yang akan dibangun nanti benar-benar relevan dengan masalah di Kampus XYZ dan didukung oleh ilmu pengetahuan terbaru. Setelah dasar pemahamannya kuat, kegiatan berlanjut ke persiapan data pada bulan Juli dan

Agustus. Di tahapan ini, buku panduan asrama dipotong dan diubah formatnya menjadi data vektor agar model kecerdasan buatan bisa membacanya secara tepat. Langkah ini sangat penting agar kecerdasan buatan tidak memberikan jawaban yang keliru kepada mahasiswa.

Setelah datanya siap, bulan Agustus dan September sepenuhnya digunakan untuk membangun program. Pada masa ini, arsitektur *Multi-Agent System* (MAS) mulai dirangkai dan dihubungkan dengan antarmuka sistem (API) kampus. Ketika program tersebut sudah berdiri utuh, tahapan selanjutnya adalah pelaksanaan eksperimen pada bulan Oktober dan November. Sistem yang telah dibuat akan diuji menggunakan riwayat keluhan asli dari mahasiswa untuk melihat apakah ia bisa bekerja dengan baik. Pada akhirnya, seluruh catatan dari pengujian tersebut dikumpulkan dan dibahas secara mendalam. Tahap analisis dan penulisan tesis ini diberikan waktu yang paling panjang, yaitu tiga bulan dari November hingga Januari, agar simpulan yang dihasilkan benar-benar akurat dan mudah dipahami oleh semua pihak.

DAFTAR PUSTAKA

- Chung, H. W., Hou, L., Longpre, S., Zoph, B., Tai, Y., Fedus, W., Li, Y., Wang, X., Dehghani, M., Brahma, S., Webson, A., Shane Gu, S., Dai, Z., Suzgun, M., Chen, X., Chowdhery, A., Castro-Ros, A., Pellat, M., Robinson, K., ... Salakhutdinov, R. (2024). Scaling Instruction-Finetuned Language Models. *Journal of Machine Learning Research*, 25(70), 1–53. <http://jmlr.org/papers/v25/23-0870.html>
- Duan, T., Zhang, Z., Lin, Z., Guo, S., Guan, X., Wu, G., Fang, Z., Meng, H., Du, X., Zhou, J. Z., Cui, H., Luo, J., & Gao, Y. (2026). LLM-Driven Stationarity-Aware Expert Demonstrations for Multi-Agent Reinforcement Learning in Mobile Systems. *IEEE Transactions on Mobile Computing*. <https://doi.org/10.1109/TMC.2026.3682994>
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., & Wang, H. (2023). *Retrieval-Augmented Generation for Large Language Models: A Survey*. <https://github.com/Tongji-KGLLM/>
- Jiang, F., Peng, Y., Dong, L., Wang, K., Yang, K., Pan, C., Niyato, D., & Dobre, O. A. (2024). Large Language Model Enhanced Multi-Agent Systems for 6G Communications. *IEEE Wireless Communications*, 31(6), 48–55. <https://doi.org/10.1109/MWC.016.2300600>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://github.com/huggingface/transformers/blob/master/>
- Li, X., Wang, S., Zeng, S., Wu, Y., & Yang, Y. (2024). A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth 2024 1:1*, 1(1), 9-. <https://doi.org/10.1007/S44336-024-00009-2>
- Liang, Y., Jiang, R., Wei, B., Tian, W., & Ma, Z. (2026). MAMoE: A Multi-Agent Mixture-of-Experts Framework for LLM-Assisted 3D Object Reconstruction and Transmission. *IEEE Transactions on Network Science and Engineering*, 13, 3862–3878. <https://doi.org/10.1109/TNSE.2025.3618771>
- Rivkin, D., Hogan, F., Feriani, A., Konar, A., Sigal, A., Liu, X., & Dudek, G. (2025). AIoT Smart Home via Autonomous LLM Agents. *IEEE Internet of Things Journal*, 12(3), 2458–2472. <https://doi.org/10.1109/IIOT.2024.3471904>
- Sun, C., Huang, S., & Pompili, D. (2025). LLM-Based Multi-Agent Decision-Making: Challenges and Future Directions. *IEEE Robotics and Automation Letters*, 10(6), 5681–5688. <https://doi.org/10.1109/LRA.2025.3562371>
- Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., & Hajishirzi, H. (2023). Self-Instruct: Aligning Language Models with Self-Generated Instructions. *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 1, 13484–13508. <https://doi.org/10.18653/V1/2023.ACL-LONG.754>
- Xu, W., Huang, C., Gao, S., & Shang, S. (2025). LLM-Based Agents for Tool Learning: A Survey. *Data Science and Engineering* 2025 10:4, 10(4), 533–563. <https://doi.org/10.1007/S41019-025-00296-9>

- Yang, Z., Peng, H., Jiang, Y., Liu, J., Luo, H., Tang, M., Li, J., & Zhang, K. (2026). LIVA: A Multi-Agent LLM-Assisted System for IoT Vulnerability Analysis. *IEEE Transactions on Dependable and Secure Computing*. <https://doi.org/10.1109/TDSC.2026.3665343>
- Yoon, S. (2025). Strategic Learning Under Linguistic and Contextual Constraints: A Theoretical Framework for LLM-Based Multi-Agent Coordination. *IEEE Access*, 13, 197053–197071. <https://doi.org/10.1109/ACCESS.2025.3628927>
- Zhong, N., Wang, Y., Xiong, R., Zheng, Y., Li, Y., Ouyang, M., Shen, D., & Zhu, X. (2024). CASIT: Collective Intelligent Agent System for Internet of Things. *IEEE Internet of Things Journal*, 11(11), 19646–19656. <https://doi.org/10.1109/JIOT.2024.3366906>